INDITEX UDC

Cátedra de IA en Algoritmos Verdes

MICROCREDENCIAL ALGORITMOS VERDES PARA LA IA: DISEÑO E IMPLEMENTACIÓN

Tema: AI, Graphics & Visual Computing: Usos de la IA para la Reconstrucción 3D. Generación de Datos Sintéticos.

Sesión: Lunes 6 de octubre de 2025

Ponente: José A. Iglesias Guitián

citic

UNIVERSIDADE DA CORUÑA

Financiado por la Unión Europea NextGenerationEU

GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA

ENIA ESTRATEGIA NACIONAL DE INTELIGENCIA ARTIFICIAL

España | digital 2026

Session Outline. Next

1. Photogrammetry and Computer Vision Fundamentals
2. **A gentle introduction to Novel View Synthesis (NVS)**
3. Synthetic image data generation for AI consumption

Novel View Synthesis (NVS). Motivation

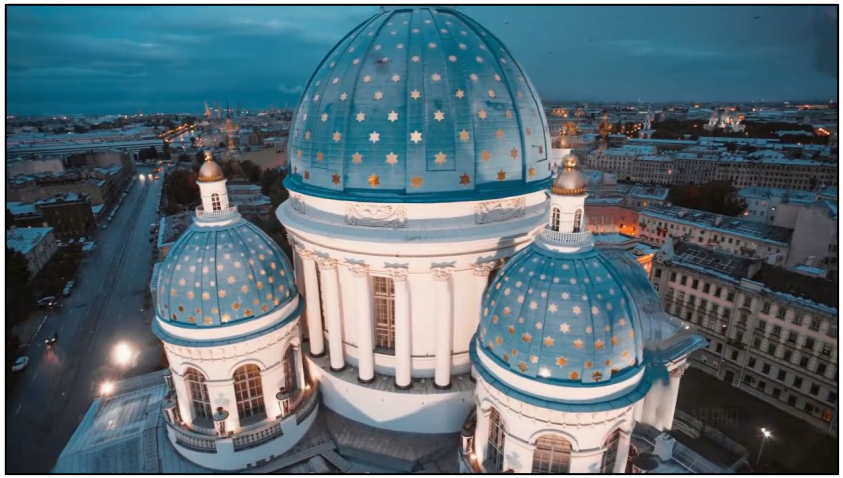
- Applications
  - AR/VR
  - Robotis
  - Digital Twins
  - Performance Capture
  - Film/Games/Entertainment

- Goal: Render unseen viewpoints from given images

Source: Song et al. CoDa-4DGS: Dynamic Gaussian Splatting with Context and Deformation Awareness for Autonomous Driving

The Novel View Synthesis (NVS) Problem

- Novel View Synthesis (NVS) is the computer vision problem of generating new images of a scene from perspectives not present in the original inputs.
- It involves creating a 3D scene representation from sparse views and then "rendering" new, photorealistic images from arbitrary viewpoints.
- This allows for applications like smooth camera movements, virtual re-shoots, and virtual house tours from just a few initial images.
- It is closely related to the field of Photogrammetry.



Camera 1

Camera 2

Novel Views

Classic Novel View Synthesis

- **Goal**: Generate new viewpoints of a scene from limited input images.

- **Before AI & Deep Learning**:
  - SfM+MVS Reconstruction
  - Mostly relied on **geometry + rendering** techniques.

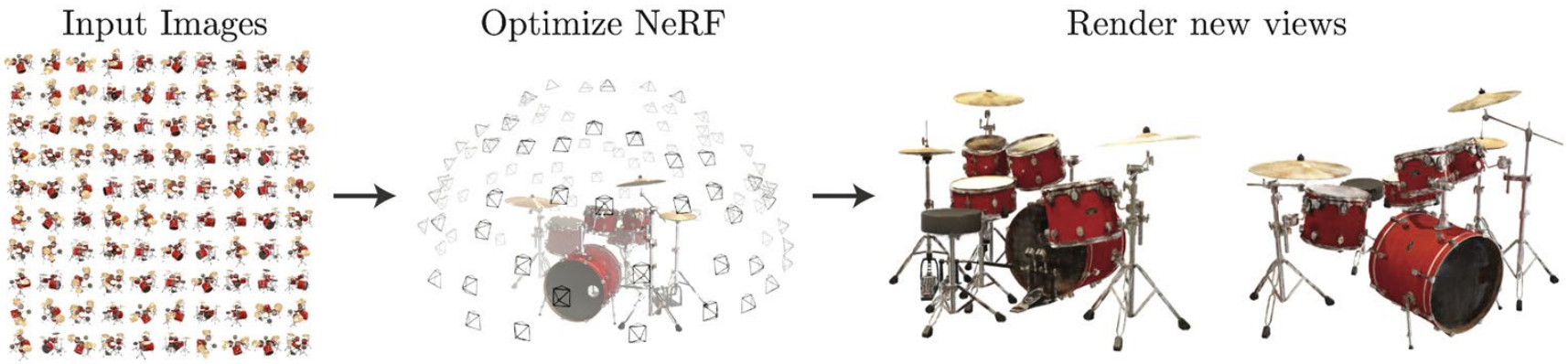
- **Key Assumption**:
  - If you know 3D structure + camera poses, you can render new views.

- **Methods**:
  - Multi-view geometry - *aims to reconstruct full 3D geometry.*
  - Structure-from-Motion (SfM) - *aims to reconstruct full 3D geometry.*
  - **Image-based rendering (IBR)** - *may or may not aim to reconstruct full 3D geometry.*

Neural Radiance Fields

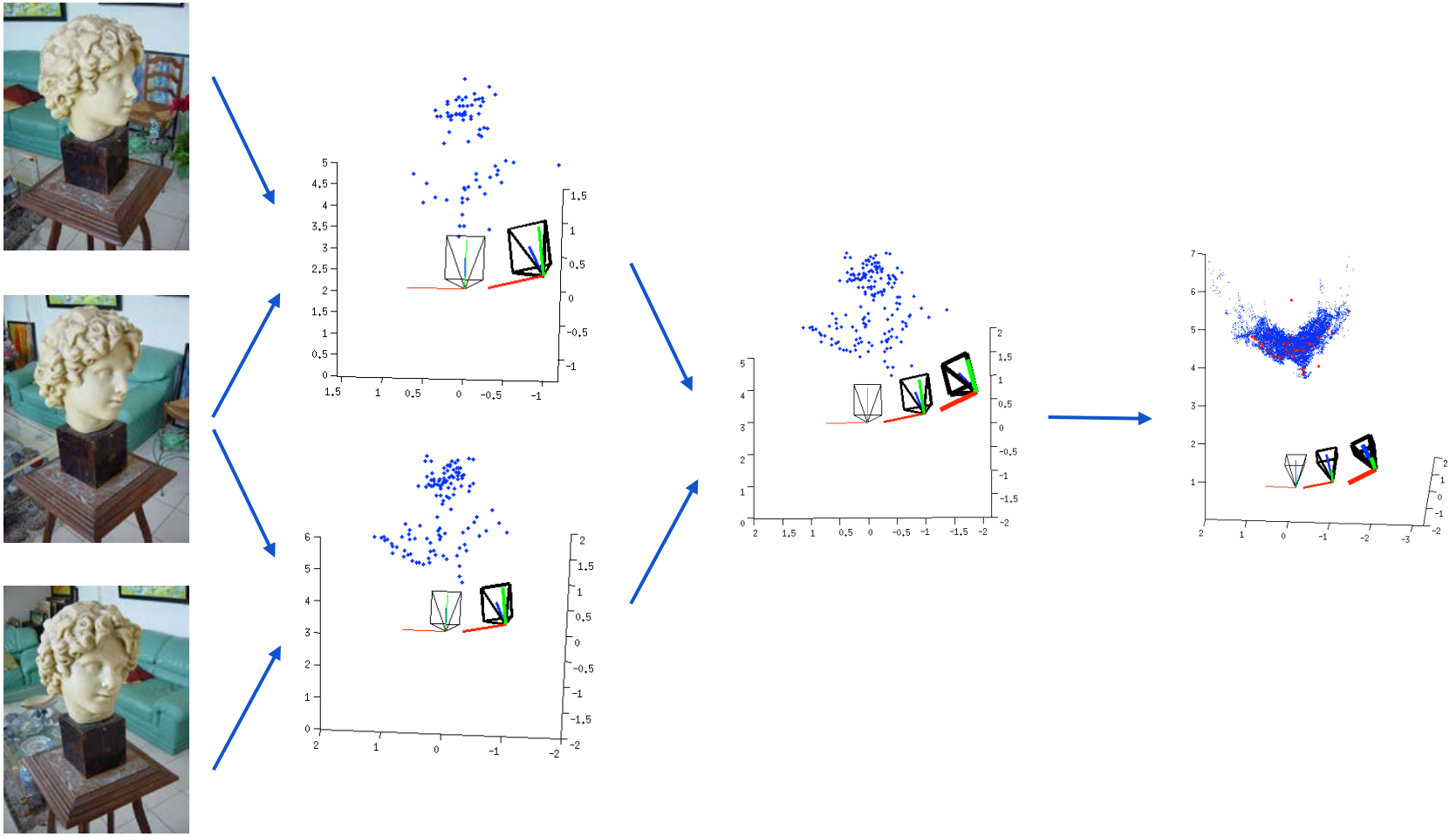
Neural Radiance Fields (NeRFs)

- **Goal**. Representing Scenes as Neural Radiance Fields for View Synthesis.
- NeRF's allowed to achieve state-of-the-art results for synthesizing novel views of complex scenes by optimizing an underlying continuous volumetric scene function using a sparse set of input views.



Source: https://www.matthewtancik.com/nerf

Incremental SfM + MVS Reconstruction Pipeline

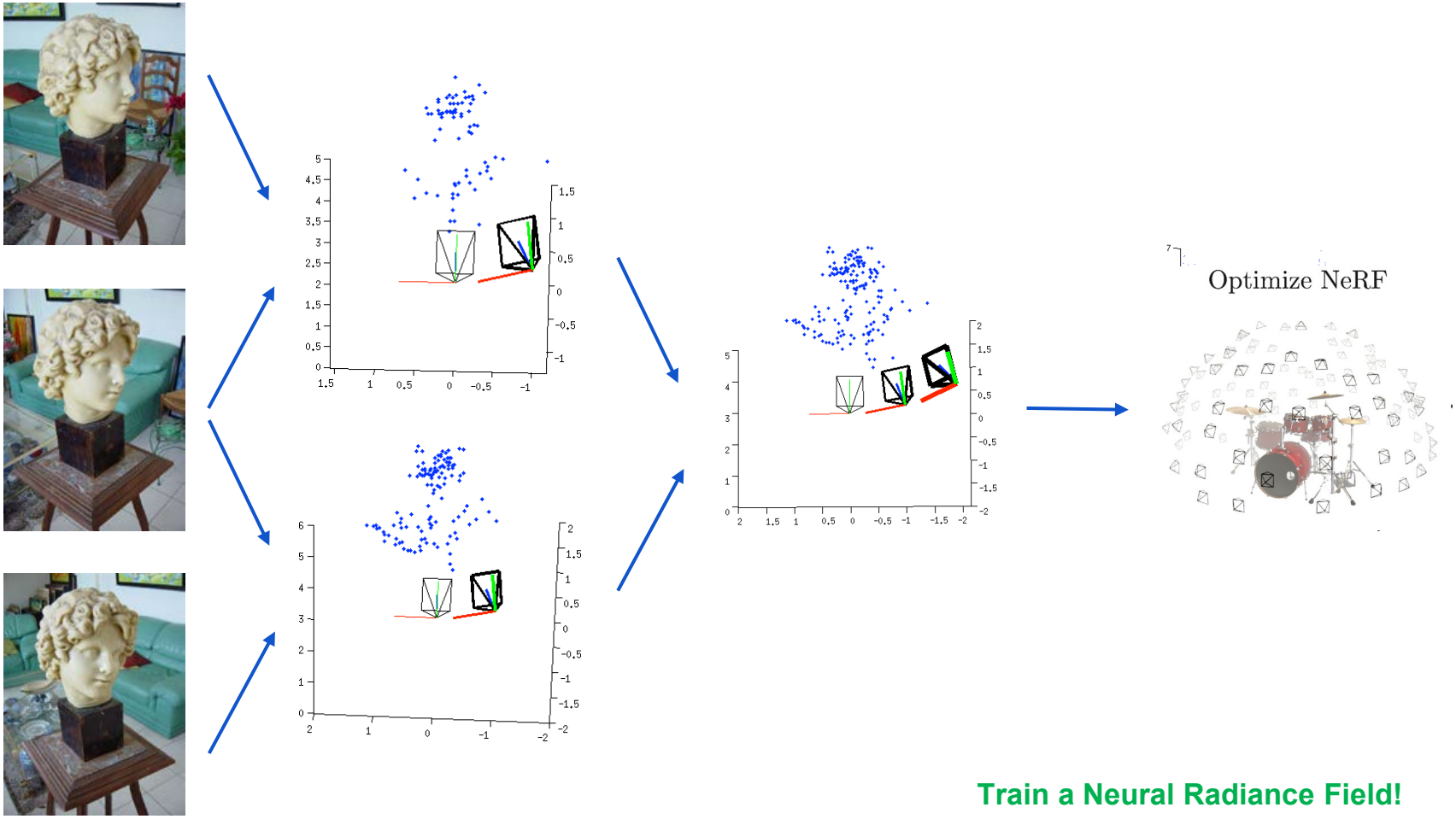


Structure from Motion (SfM) **Multi-view Stereo (MVS)**

Source: Robot Vision Group. National Laboratory of Pattern Recogntion Institute of Automation, Chinese Academy of Sciences

INDITEX UDC
Cátedra de IA en Algoritmos Verdes

Incremental SfM + IBR Reconstruction Pipeline



Train a Neural Radiance Field!

Structure from Motion (SfM)

~~Multi-view Stereo (MVS)~~

Source: Robot Vision Group. National Laboratory of Pattern Recogntion Institute of Automation, Chinese Academy of Sciences

Neural Radiance Fields (NeRFs). Key idea

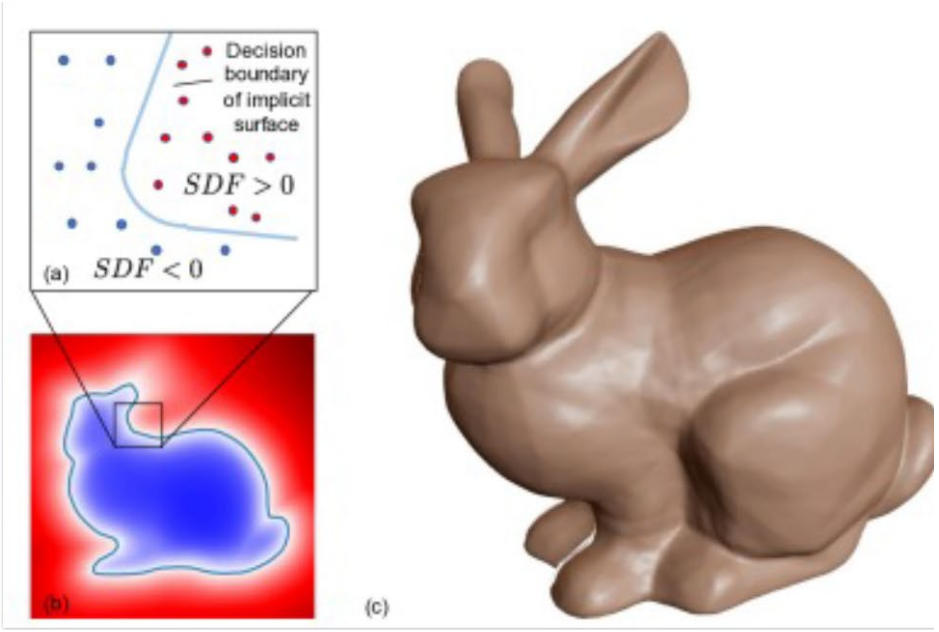
Source: NeRF- Neural Radiance Fields https://www.youtube.com/watch?v=JuH79E8rdKc

Neural Radiance Fields (NeRFs). Related work

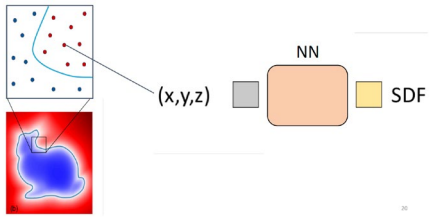
- Surface representation: Signed Distance Function (SDF) via level sets.

SDF(X) = 0, when X is on the surface.
SDF(X) > 0, when X is outside the surface
SDF(X) < 0, when X is inside the surface

Note: SDF is an implicit representation!
Suitable for neural networks but hard to import inside existing graphics software.



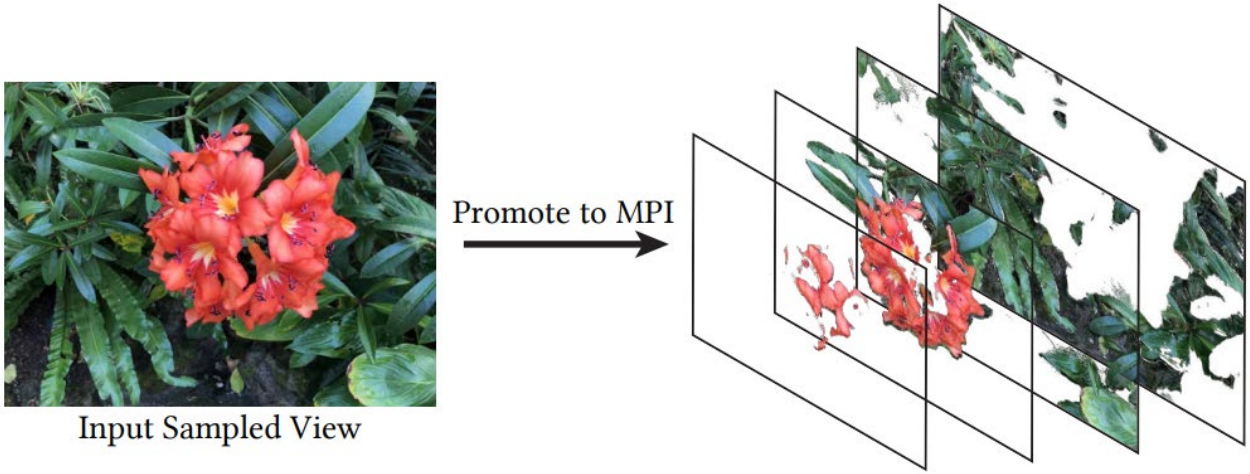
Deep SDF: Use a neural network (co-ordinate based MLP) to represent the SDF function.



Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Neural Radiance Fields (NeRFs). Related work

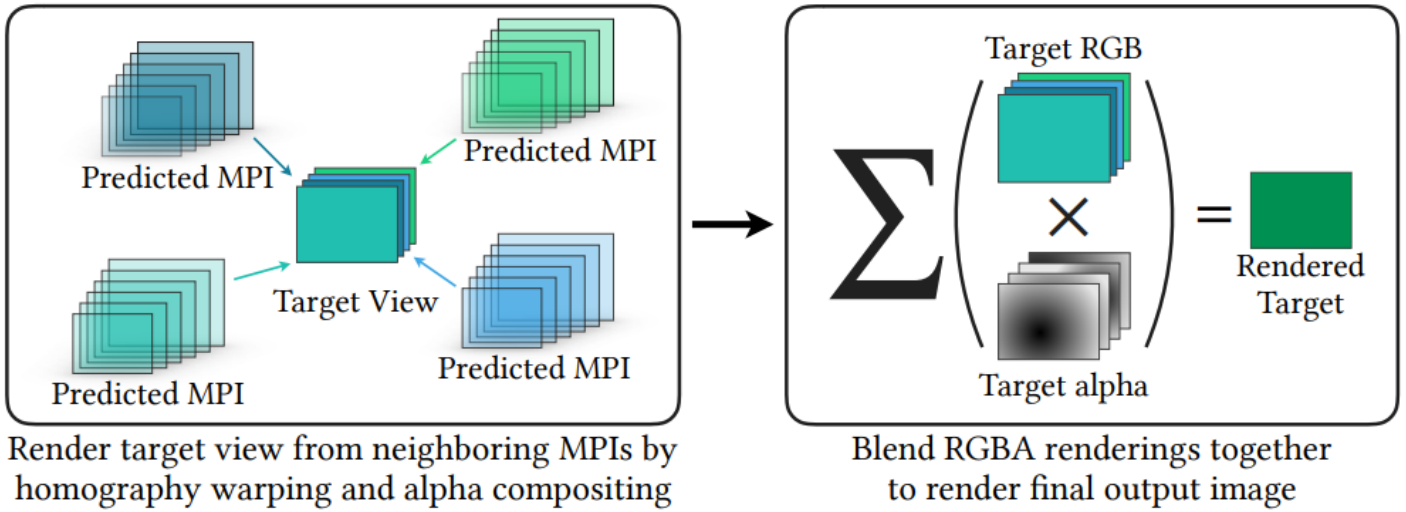
- Local Light Field Fusion:
  - Promote input views to **Multi-Planar Image (MPI) representations** consisting of $\mathrm{RGB}\alpha$ planes
    - MPI planes are regularly sampled within the input view's camera frustum
    - It uses a trained 3D convolutional network
    - Each MPI can render continuously-valued novel views within a local neighborhood by **alpha compositing** color along rays into the novel view's camera



Source: Mildenhall et al. Local Light Field Fusion: Practical View Synthesis with Prescriptive Sampling Guidelines

Neural Radiance Fields (NeRFs). Related work

- Local Light Field Fusion:
  - Promote input views to **Multi-Planar Image (MPI) representations** consisting of RGBα planes
    - MPI planes are regularly sampled within the input view's camera frustum
    - It uses a trained 3D convolutional network
    - Each MPI can render continuously-valued novel views within a local neighborhood by **alpha compositing** color along rays into the novel view's camera



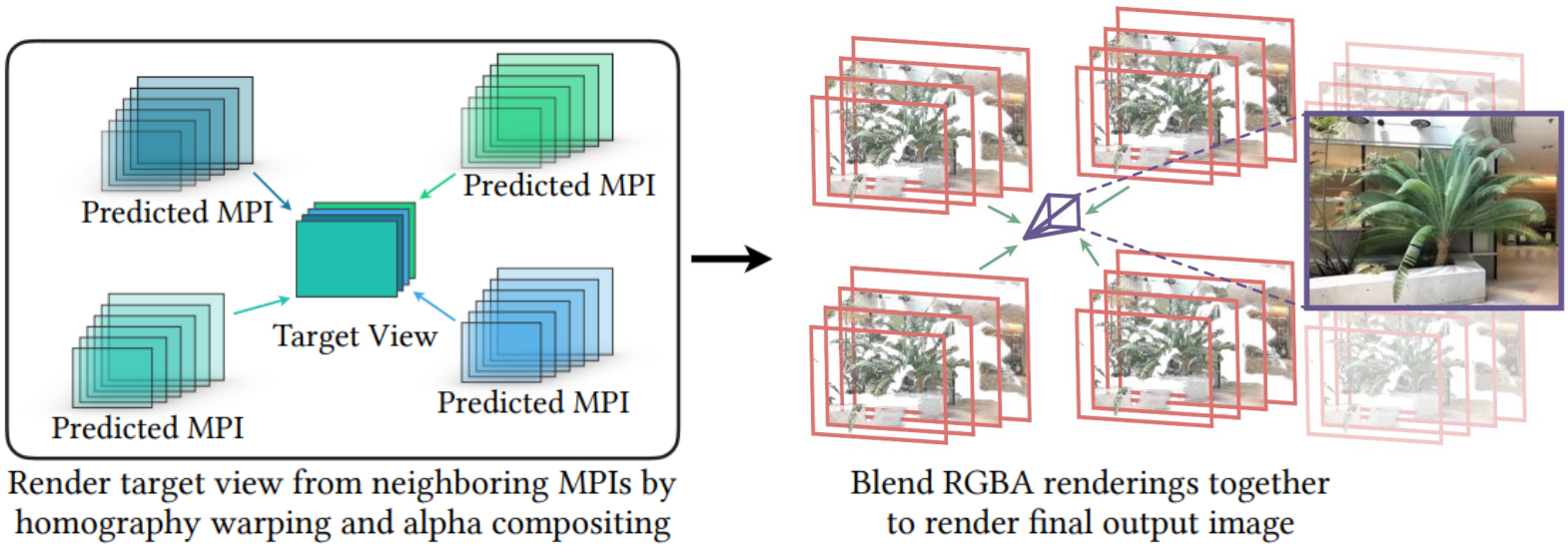
Render target view from neighboring MPIs by homography warping and alpha compositing

Blend RGBA renderings together to render final output image

Source: Mildenhall et al. Local Light Field Fusion: Practical View Synthesis with Prescriptive Sampling Guidelines

Neural Radiance Fields (NeRFs). Related work

- Local Light Field Fusion:
  - Promote input views to **Multi-Planar Image (MPI) representations** consisting of RGBα planes
    - MPI planes are regularly sampled within the input view's camera frustum.
    - It uses a trained 3D convolutional network
    - Each MPI can render continuously-valued novel views within a local neighborhood by **alpha compositing** color along rays into the novel view's camera (~ **volume rendering**).



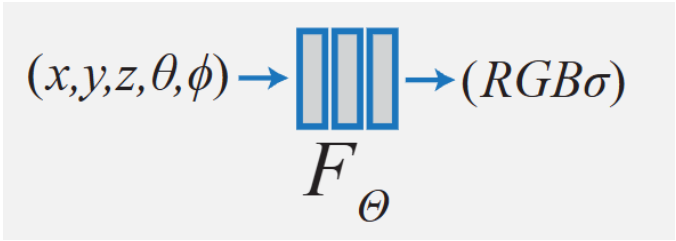
Render target view from neighboring MPIs by homography warping and alpha compositing

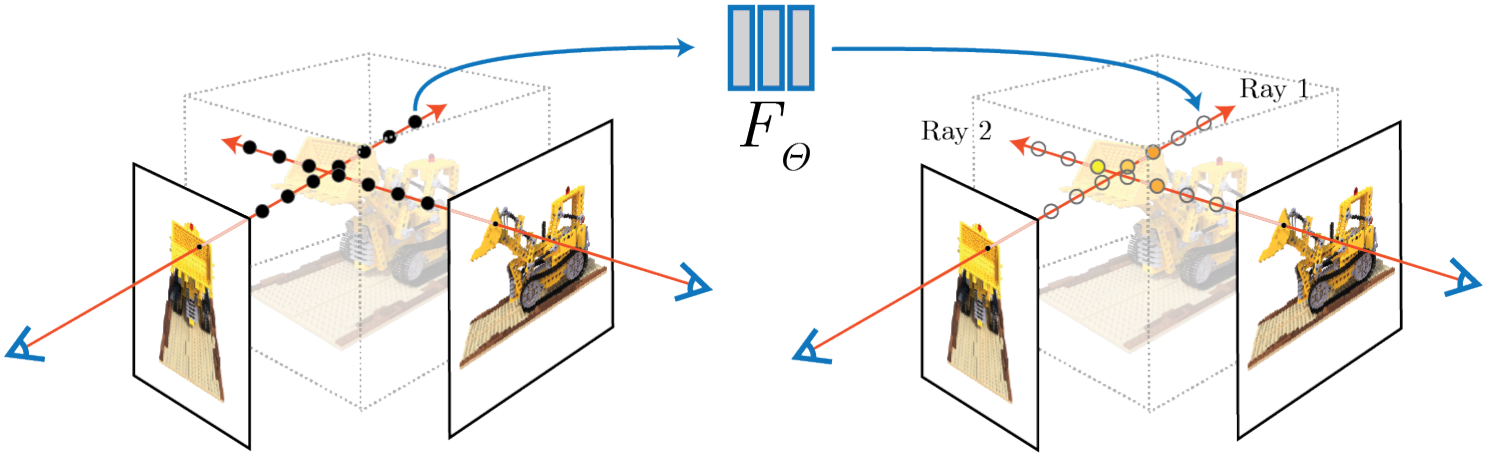
Blend RGBA renderings together to render final output image

Source: Mildenhall et al. Local Light Field Fusion: Practical View Synthesis with Prescriptive Sampling Guidelines

Neural Radiance Fields (NeRFs). Key idea

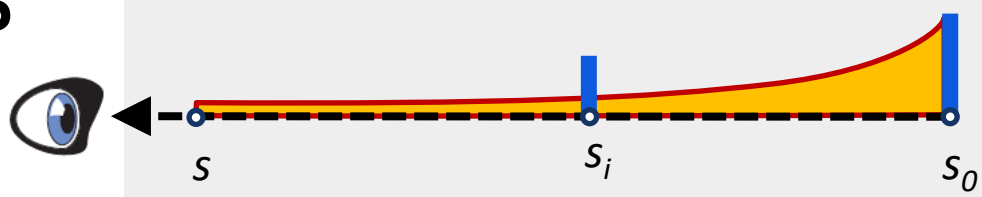
- Represent a scene using a **MLP (Multi-Layer Perceptron)**
  - MLP's are fully-connected, non-convolutional, **deep networks.**
- The inputs is 5D coordinate: spatial location (x, y, z) and viewing dir (θ, ϕ).
- The output is the **volume density** (σ) and **view-dependent emitted radiance.**





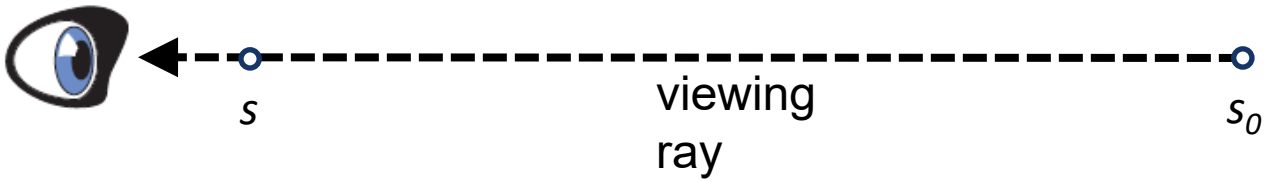
Volume Rendering

Ray Integration



Volume Rendering. Ray Integration

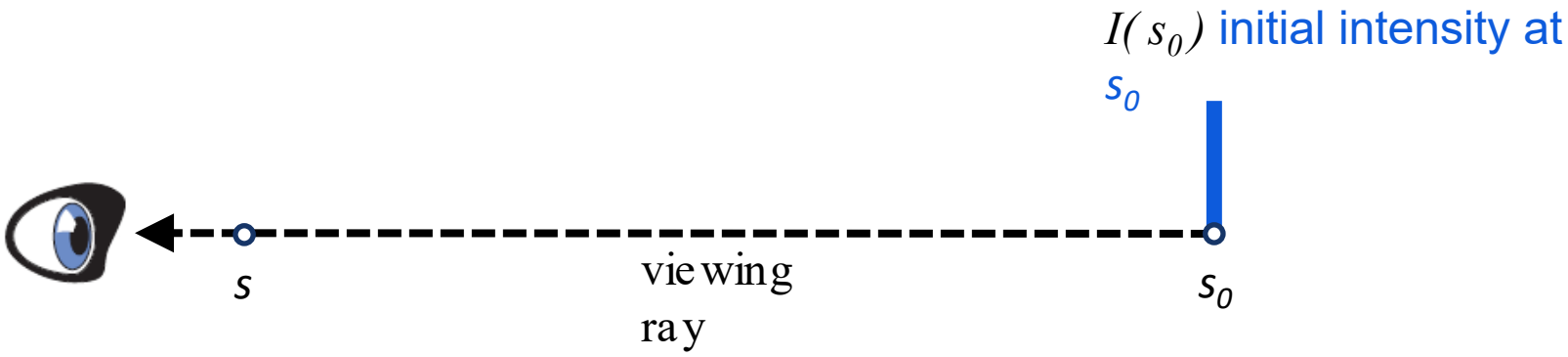
- Volume Rendering using ray-marching:



$I(s)$

Volume Rendering. Ray Integration

- Volume Rendering using ray-marching:



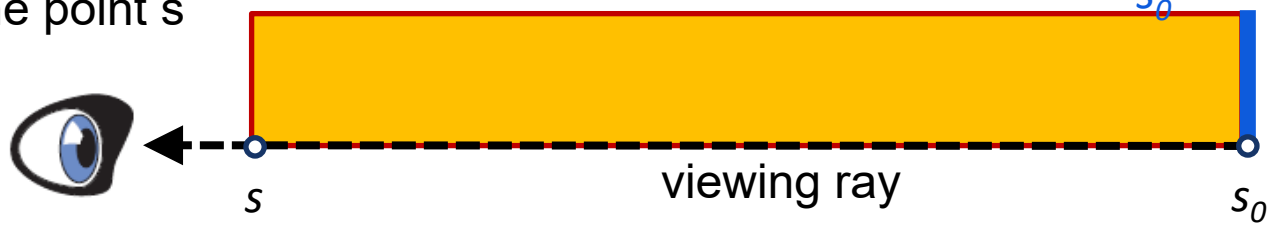
$$I(s) = I(s_0)$$

Volume Rendering. Ray Integration

- Volume Rendering using ray-marching:

Without absorption all the initial radiant energy would reach the point s

$I(s_0)$ initial intensity at s_0

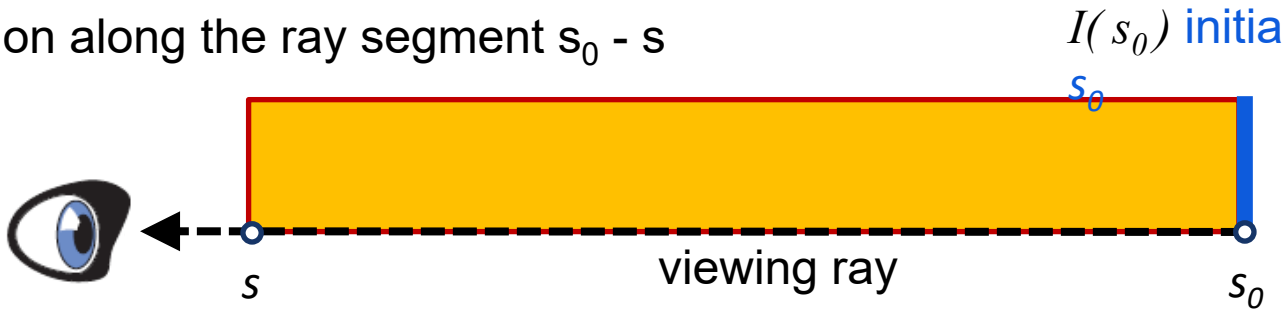


$$I(s) = I(s_0)$$

Volume Rendering. Ray Integration

- Volume Rendering using ray-marching:

Absorption along the ray segment s_0 - s



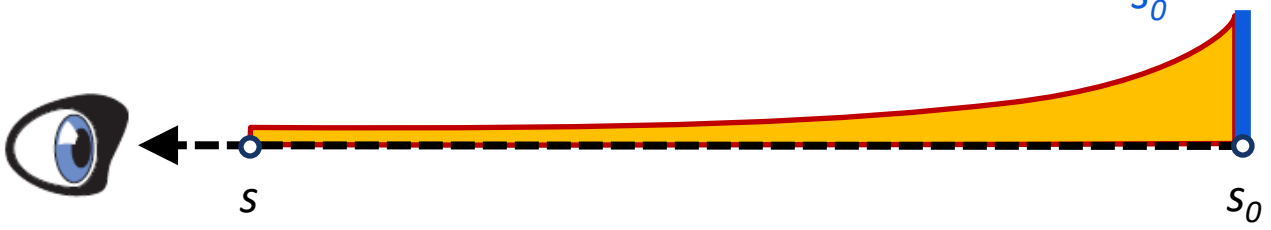
$$I(s) = I(s_0)\, e^{-\tau(s_0,s)}$$

Volume Rendering. Ray Integration

- Volume Rendering using ray-marching:

Absorption along the ray segment s_0 - s

$I(s_0)$ initial intensity at s_0



$$I(s) = I(s_0)\, e^{-\tau(s_0,s)}$$

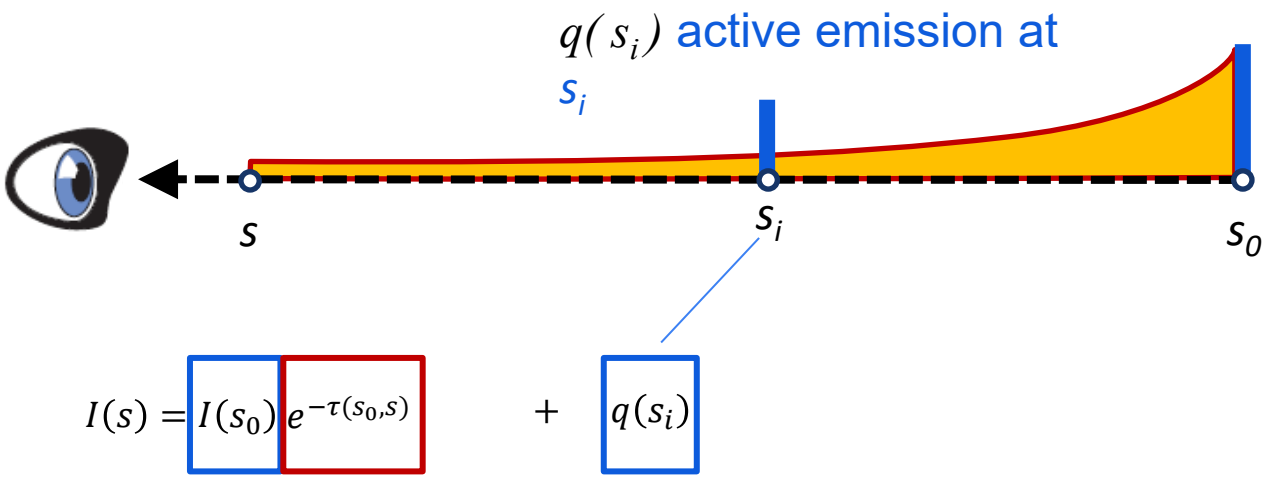
Optical thickness

Extinction τ
Absorption κ

$$\tau(s_1, s_2) = \int_{s_1}^{s_2} \kappa(s)ds$$

Volume Rendering. Ray Integration

- Volume Rendering using ray-marching:

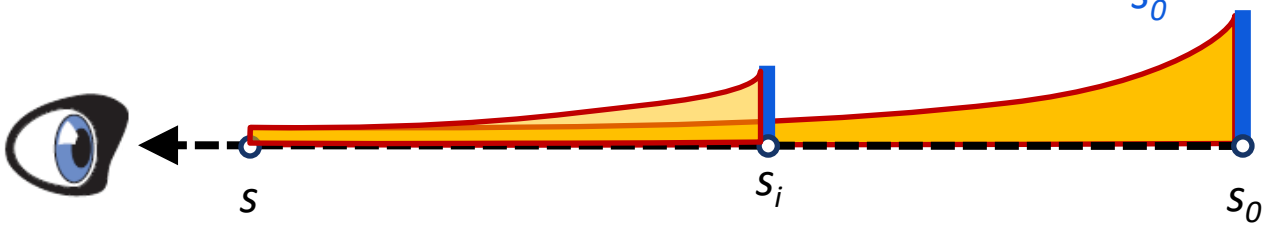


Volume Rendering. Ray Integration

- Volume Rendering using ray-marching:

Absorption along the ray segment s_i - s

$I(s_0)$ initial intensity at s_0



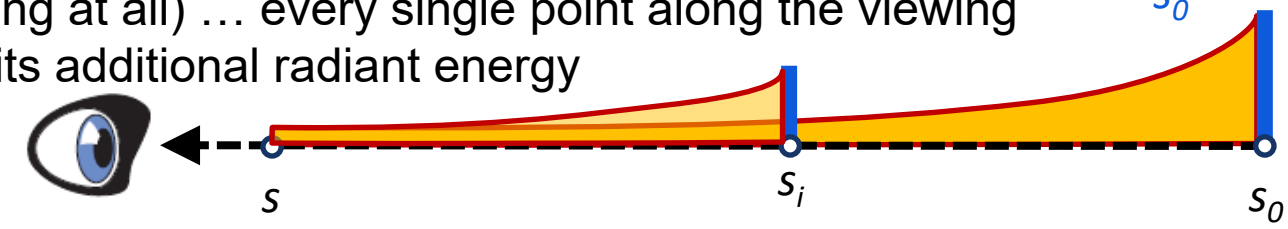
$$I(s) = I(s_0)\, e^{-\tau(s_0,s)} + q(s_i)\, e^{-\tau(s_i,s)} ds_i$$

Volume Rendering. Ray Integration

- Volume Rendering using ray-marching:

Assuming an emission-absorption model (with no scattering at all) … every single point along the viewing ray emits additional radiant energy

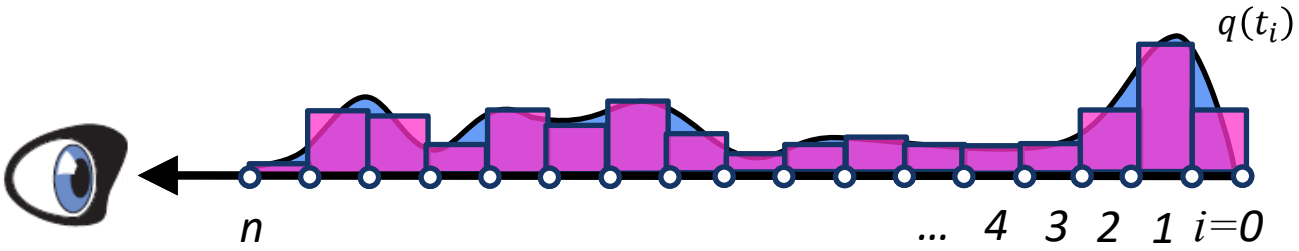
$I(s_0)$ initial intensity at s_0

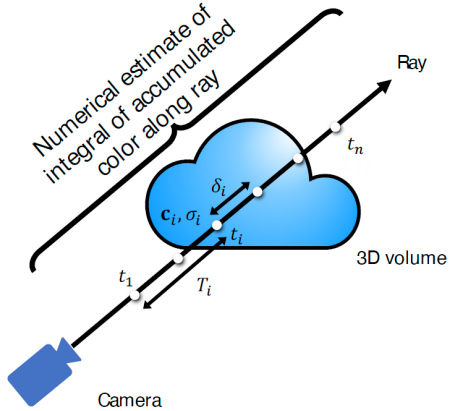


$$I(s) = I(s_0)\, e^{-\tau(s_0,s)} + \int_{s_0}^{s} q(s_i) e^{-\tau(s_i,s)} ds_i$$

Volume Rendering. Numerical Solution

- Approximate Integral by Riemann Sum





$$\mathbf{c} \approx \sum_{i=1}^{n} T_i \alpha_i \mathbf{c}_i \qquad T_i = \prod_{j=1}^{i-1} (1 - \alpha_j) \qquad \alpha_i = 1 - \exp(-\sigma_i \delta_i)$$

final rendered color along ray

weights

colors

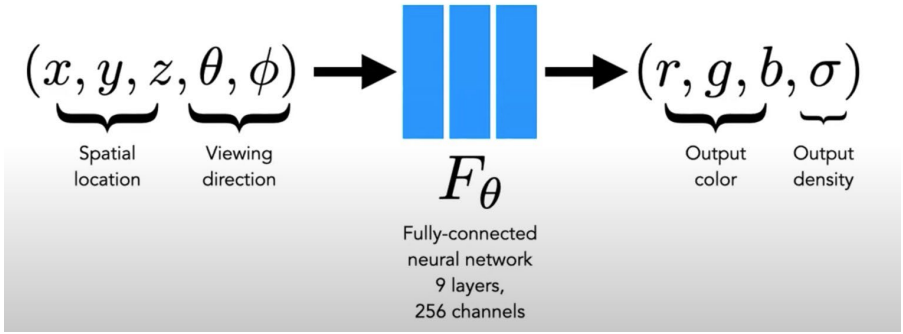
transmittance represents the light blocked earlier along the ray

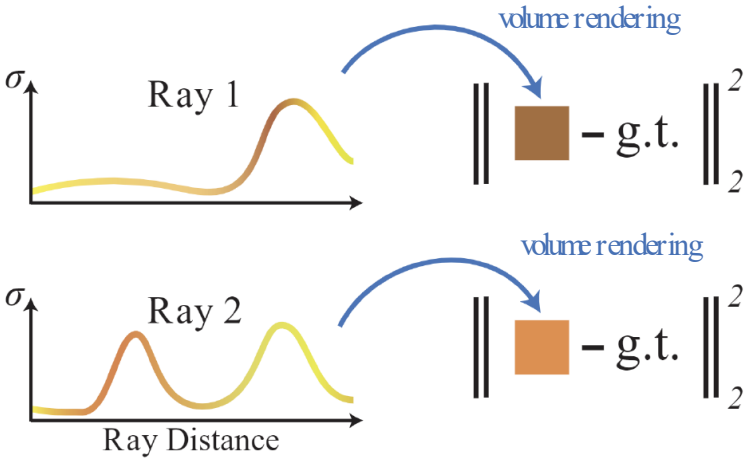
transparency depends on density

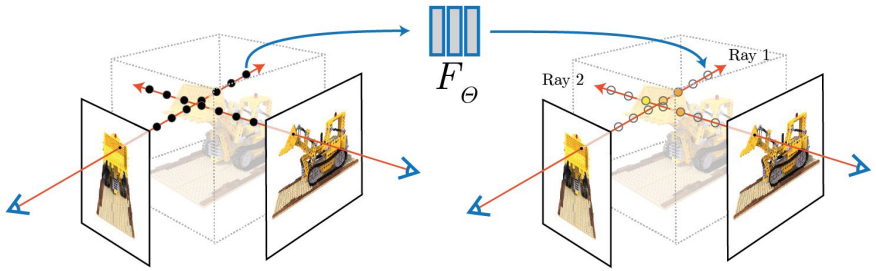
Neural Radiance Fields (continue)

Neural Radiance Fields (NeRFs). Rendering NeRF's

- NeRF could be seen as (5D) coordinate-based modeling of RGB and Densities.
- The technique used to compose and project NeRF's is **Volume Rendering**.
  - Volume rendering is differentiable, thus it is well-suited for optimizing the NeRF's representation.

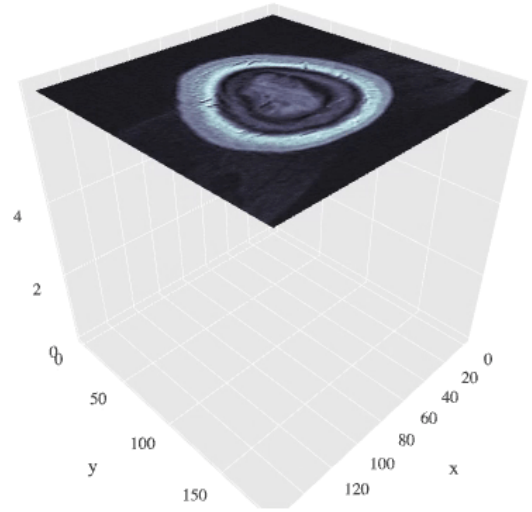






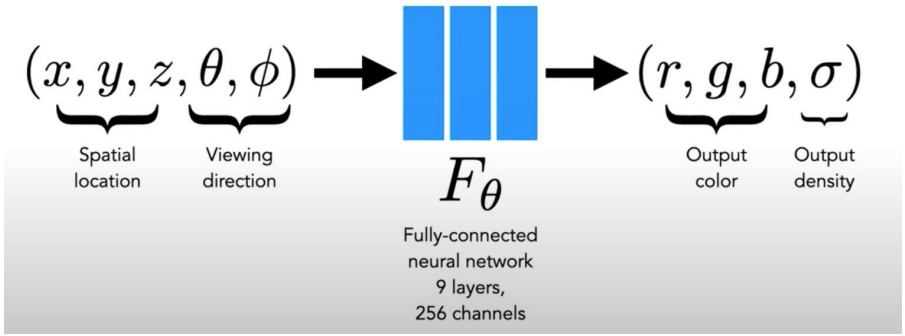
Source: Source: NeRF- Neural Radiance Fields https://www.youtube.com/watch?v=JuH79E8rdKc

Neural Radiance Fields (NeRFs). Storing 5D (θ) radiance (r, g, b, σ)

(x, y, z) →  → (r, g, b, σ)

No view-dependent effects!

Versus



+ View-dependent effects!

Toy Problem (MLPs)

Toy problem: storing a 2D image using a neural network (MLP)

- Images are typically stored as a 2D grid of (r, g, b) values.

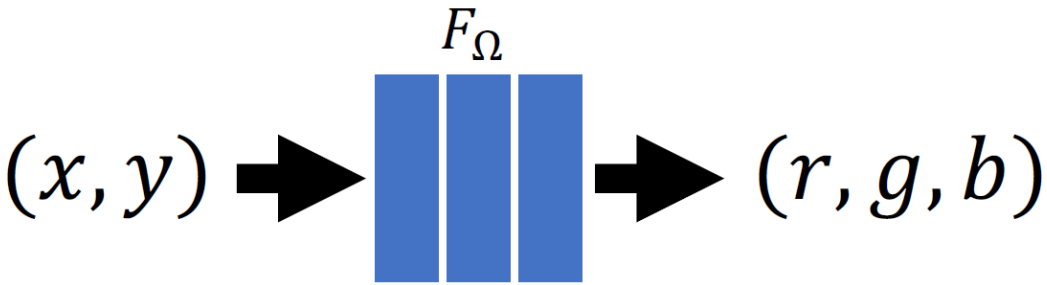
(x, y)

(r, g, b)

Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

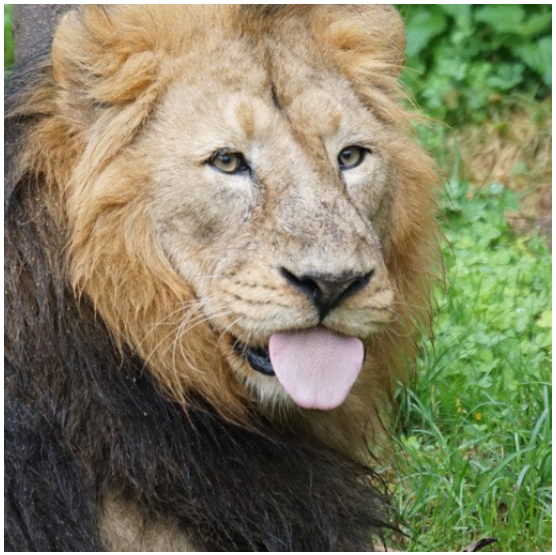
Toy problem: storing a 2D image using a neural network (MLP)

- What if we train a simple fully-connected neural network (MLP) instead?



Toy problem: storing a 2D image using a neural network (MLP)

- What if we train a simple fully-connected neural network (MLP) instead?
  - The naive approach fails!



Ground truth image

NN output fit with gradient descent

Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Fundamental Problem of Coordinate-based MLP's

- **Problem**:
  - Standard coordinate-based MLP's are bad at representing high-frequency funtions.

- **Solution**:
  - Pass input coordinates through a high-frequency mapping first …

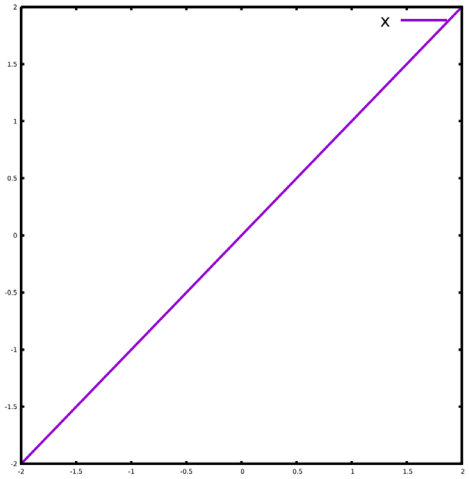
Example mapping:
"positional encoding"

$$\begin{pmatrix} \sin(\mathbf{v}), \cos(\mathbf{v}) \\ \sin(2\mathbf{v}), \cos(2\mathbf{v}) \\ \sin(4\mathbf{v}), \cos(4\mathbf{v}) \\ \dots \\ \sin(2^{L-1}\mathbf{v}), \cos(2^{L-1}\mathbf{v}) \end{pmatrix}$$

Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

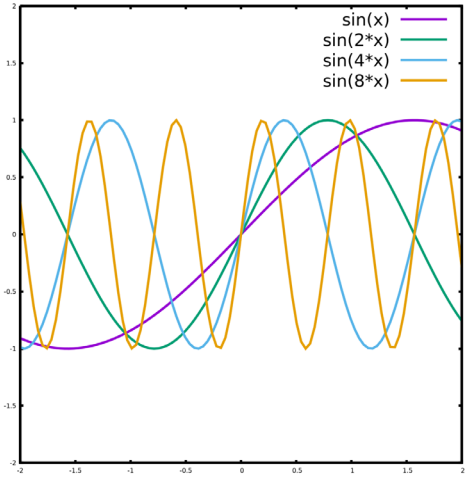
Fundamental Problem of Coordinate-based MLP's

- Raw-encoding of a number **x**



- Positional encoding of a number **x**

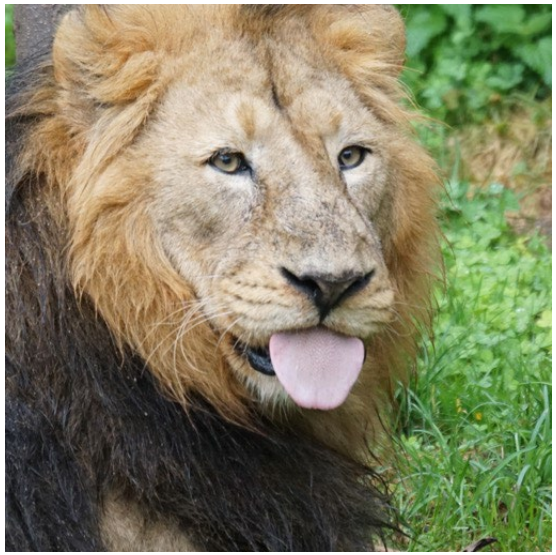
$$\begin{pmatrix} \sin(\mathbf{v}), \cos(\mathbf{v}) \\ \sin(2\mathbf{v}), \cos(2\mathbf{v}) \\ \sin(4\mathbf{v}), \cos(4\mathbf{v}) \\ \dots \\ \sin(2^{L-1}\mathbf{v}), \cos(2^{L-1}\mathbf{v}) \end{pmatrix}$$



Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

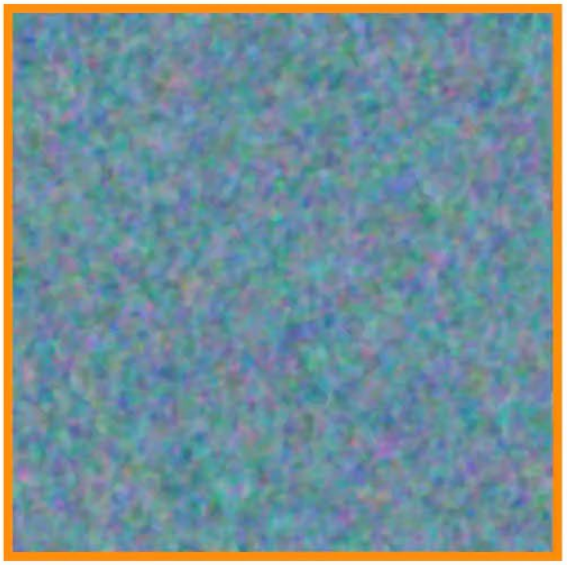
Toy problem: storing a 2D image using a neural network (MLP)

- Problem solved!



Ground truth image

NN output w/o high
– freq. mapping

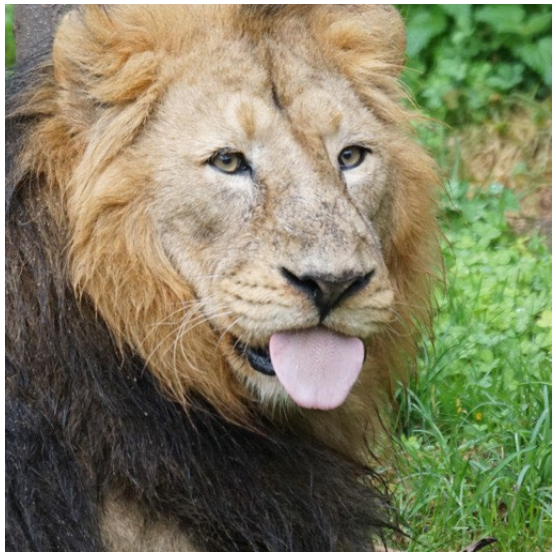


NN output with high
– freq. mapping

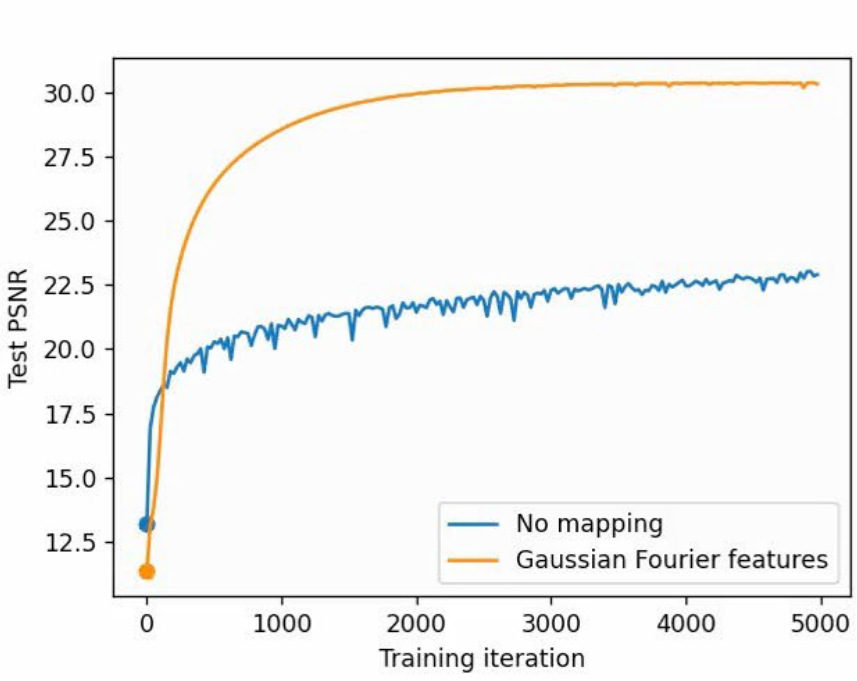
Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Toy problem: storing a 2D image using a neural network (MLP)

- Problem solved!



Ground truth image



Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Toy exercise: storing a 2D image using a neural network (MLP)

- Plain coords
- Positional Encoding
- Improving network capacity

Target

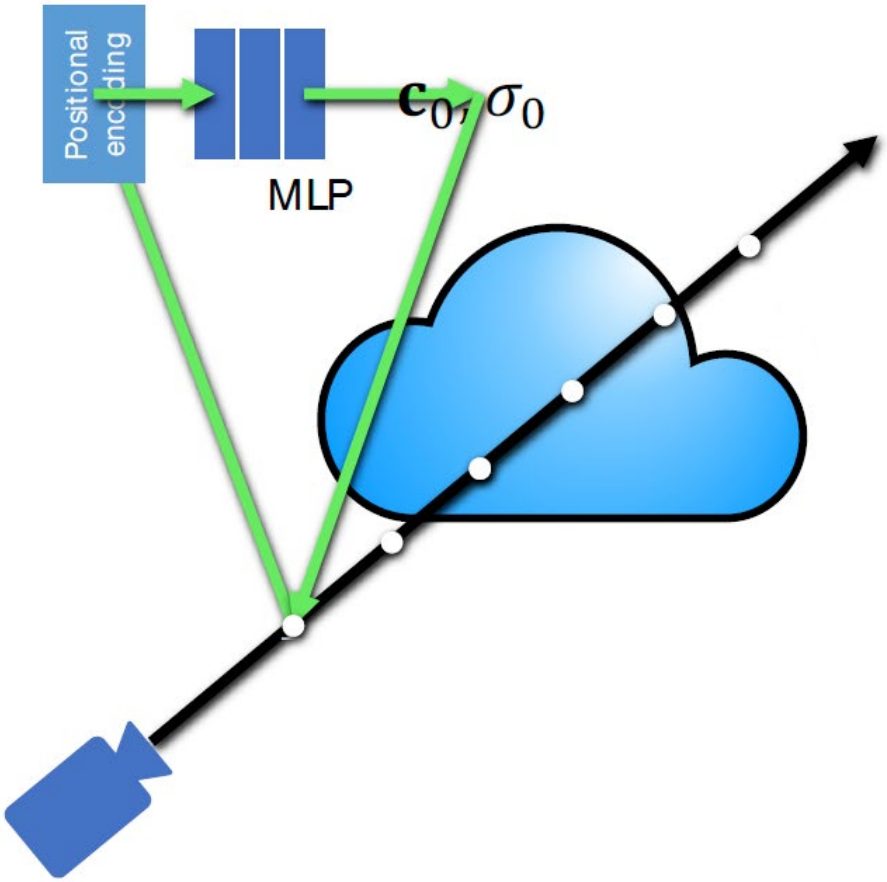
Plain

Positional Enc.

Positional Enc.L20H256

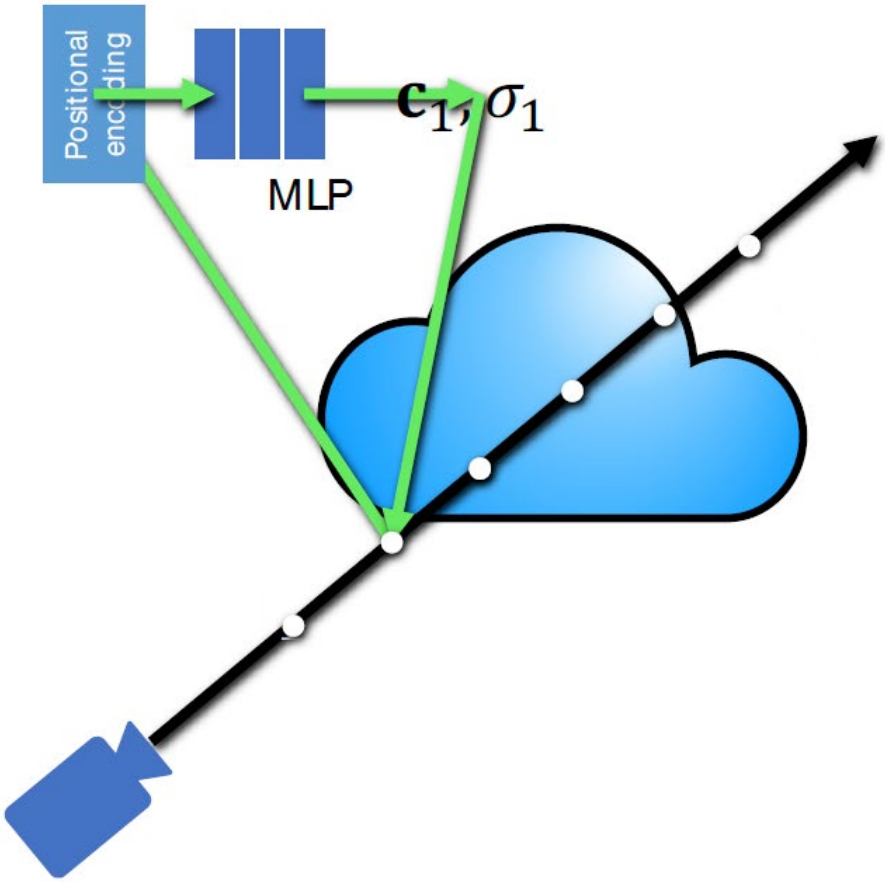
Neural Radiance Fields (continue)

Neural Radiance Fields (NeRFs) = Volume rendering + coord.-based MLP



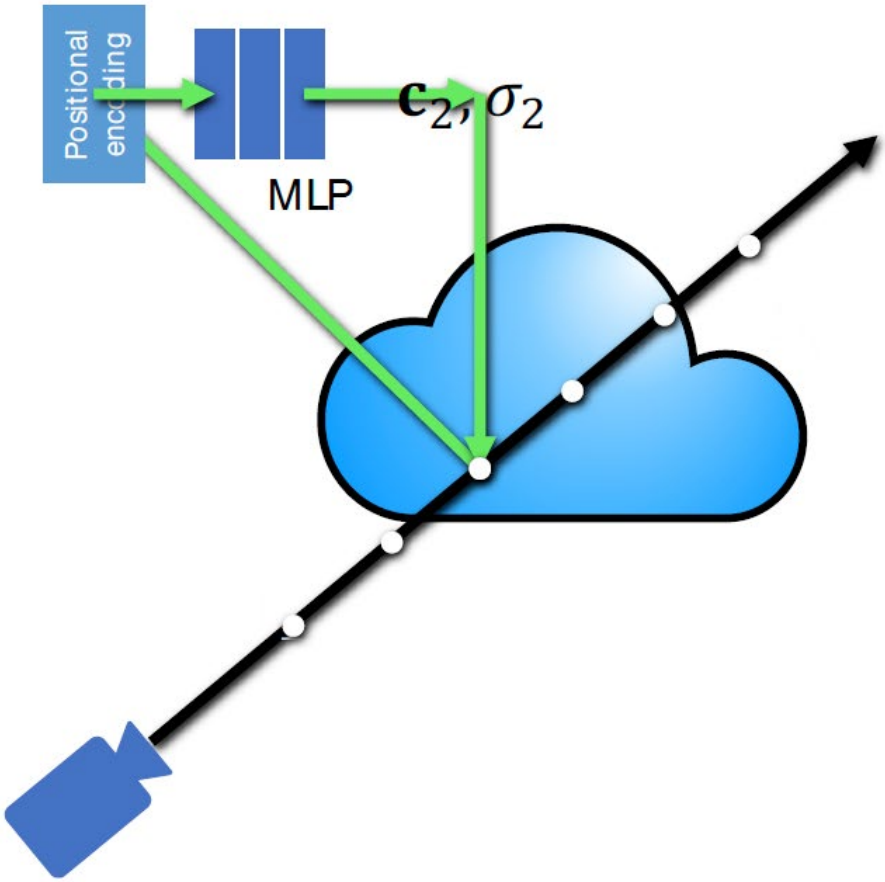
Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Neural Radiance Fields (NeRFs) = Volume rendering + coord.-based MLP



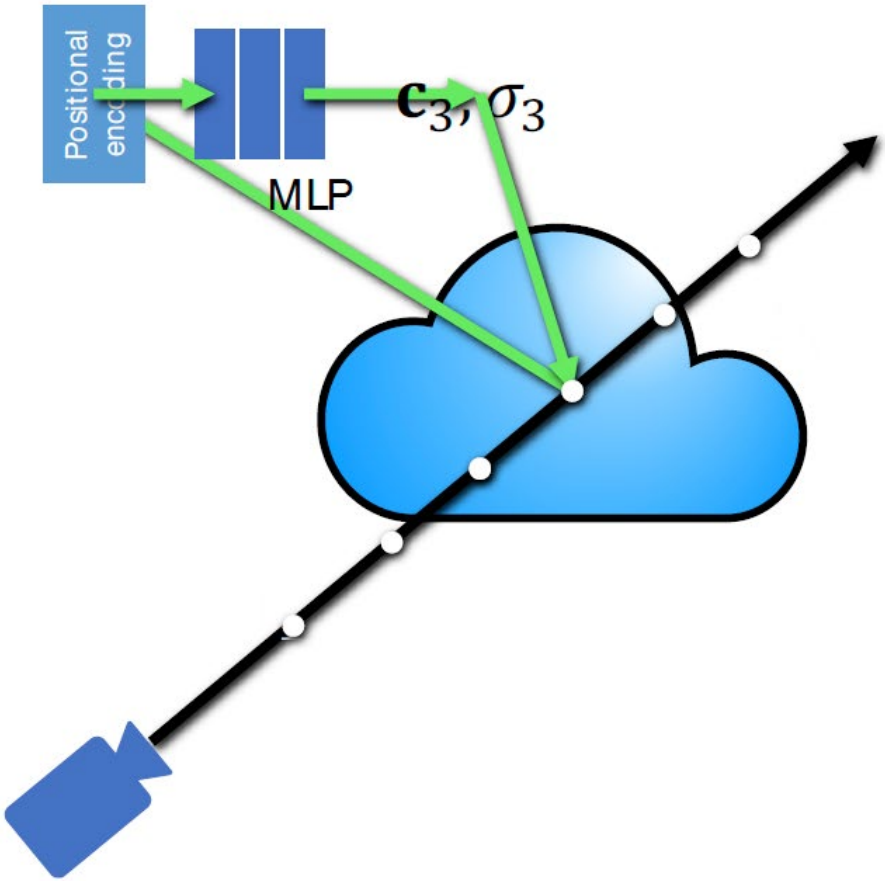
Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Neural Radiance Fields (NeRFs) = Volume rendering + coord.-based MLP



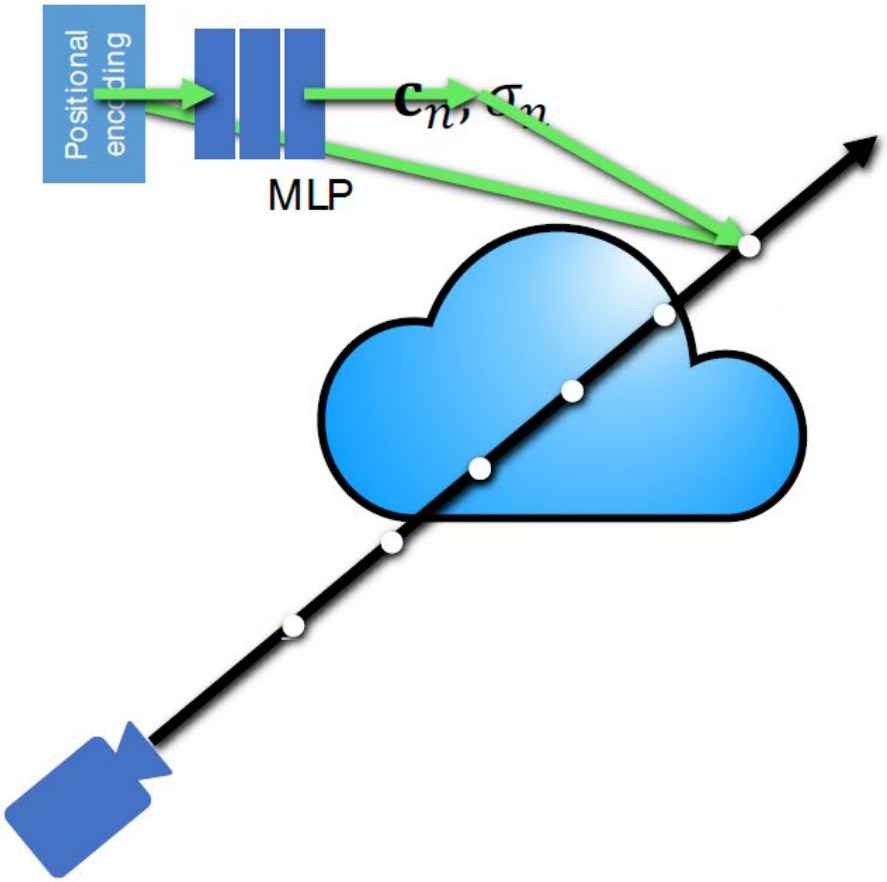
Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Neural Radiance Fields (NeRFs) = Volume rendering + coord.-based MLP



Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

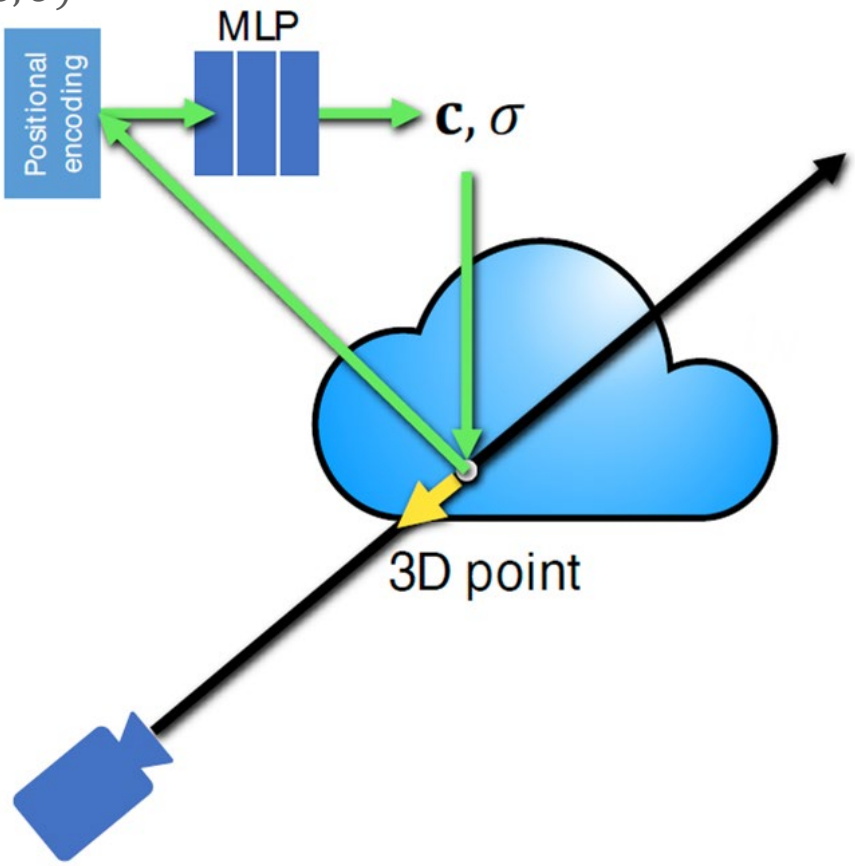
Neural Radiance Fields (NeRFs) = Volume rendering + coord.-based MLP



Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

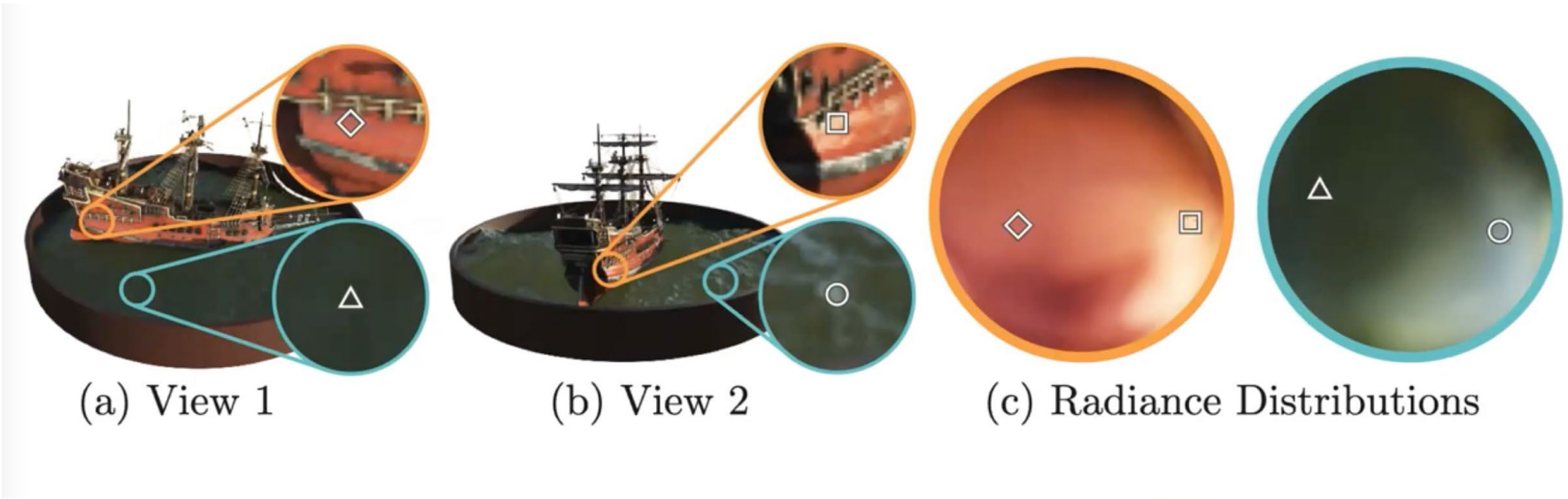
Neural Radiance Fields (NeRFs) = Volume rendering + coord.-based MLP

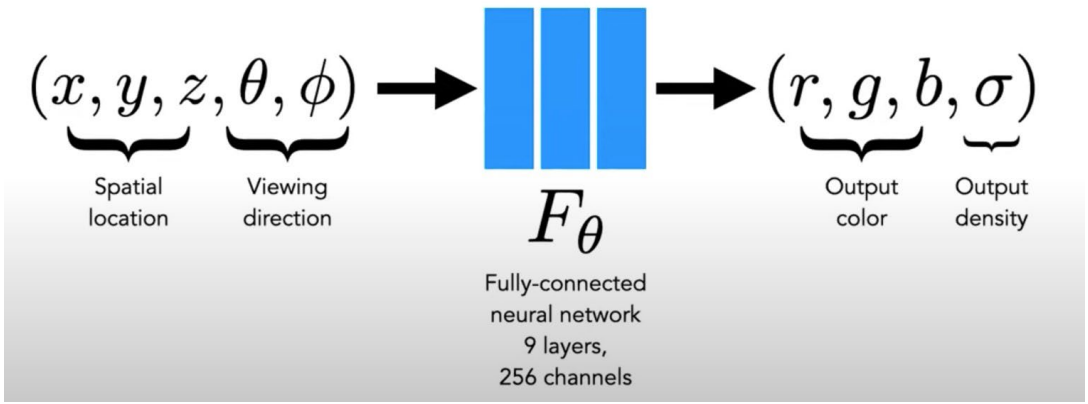
- Include **view (ray) direction** in the input to the MLP …
  - ➔ Allows to capture and render **view-dependent effects**, e.g. shiny surfaces, transparency, …
  - ➔ Final result is $(\boldsymbol{c}, \boldsymbol{\sigma})$

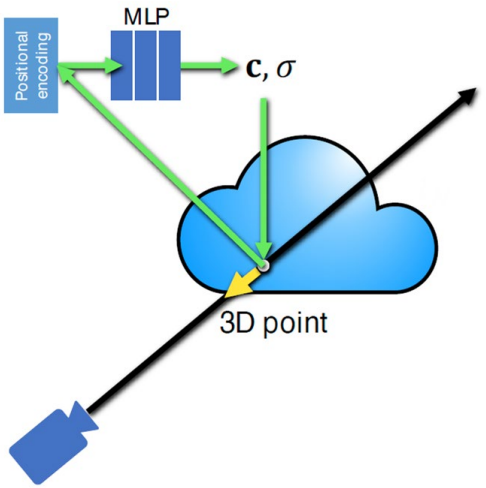


Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Neural Radiance Fields (NeRFs). View-dependent effects



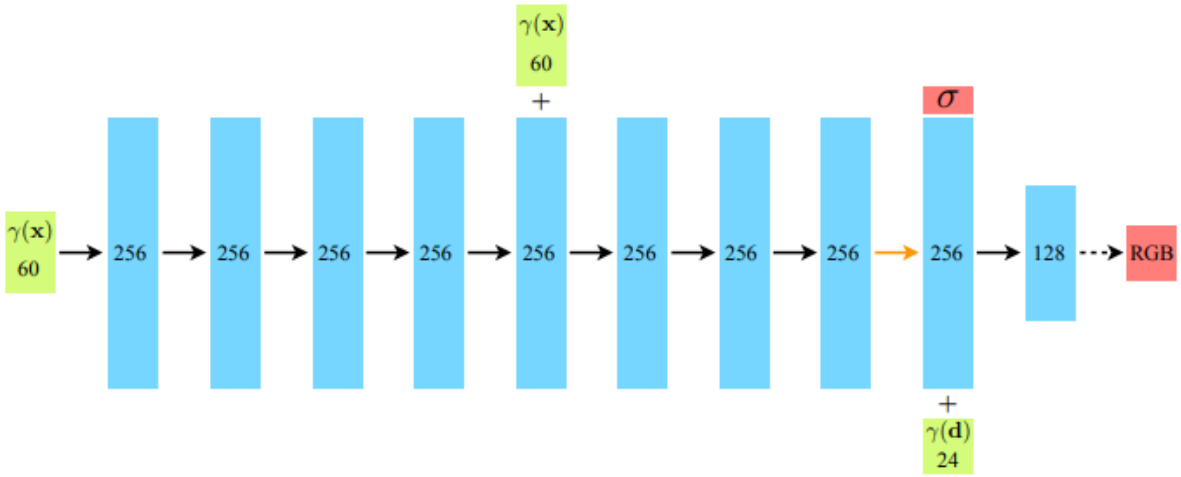




Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Training Neural Radiance Fields (NeRFs). Architecture

- Fully-connected NN architecture:



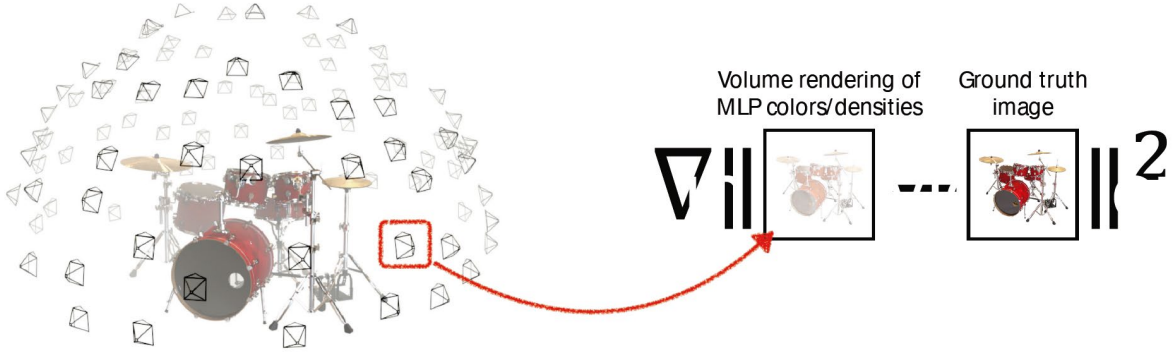
- Loss function:

$$\mathcal{L} = \sum_{\mathbf{r}\in\mathcal{R}} \left[\left\| \hat{C}_c(\mathbf{r}) - C(\mathbf{r}) \right\|_2^2 + \left\| \hat{C}_f(\mathbf{r}) - C(\mathbf{r}) \right\|_2^2 \right]$$

Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Training Neural Radiance Fields (NeRFs). Training

- Optimization problem: $$\min_{\theta} \sum_{i} || \text{render}_i(F_\theta) - I_i||^2$$



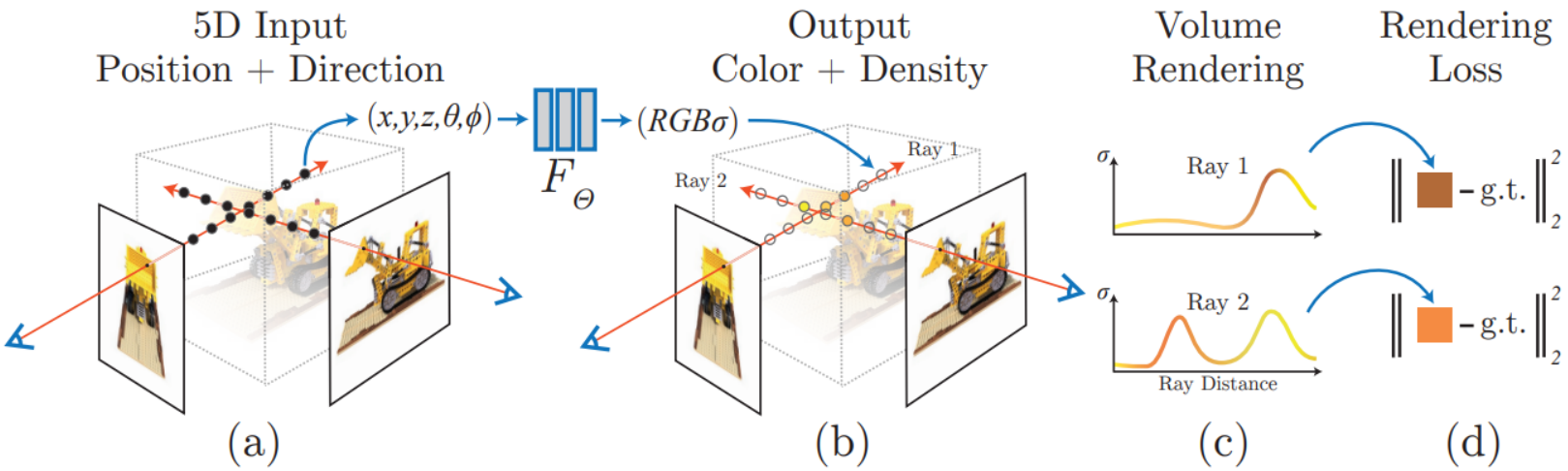
- Loss function:

$$\mathcal{L} = \sum_{\mathbf{r}\in\mathcal{R}} \left[\left\| \hat{C}_c(\mathbf{r}) - C(\mathbf{r}) \right\|_2^2 + \left\| \hat{C}_f(\mathbf{r}) - C(\mathbf{r}) \right\|_2^2 \right]$$

Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Training Neural Radiance Fields (NeRFs). Training

- Optimization problem: $$\min_{\theta} \sum_{i} || \text{render}_i(F_\theta) - I_i ||^2$$



- Loss function:

$$\mathcal{L} = \sum_{\mathbf{r}\in\mathcal{R}} \left[\left\| \hat{C}_c(\mathbf{r}) - C(\mathbf{r}) \right\|_2^2 + \left\| \hat{C}_f(\mathbf{r}) - C(\mathbf{r}) \right\|_2^2 \right]$$

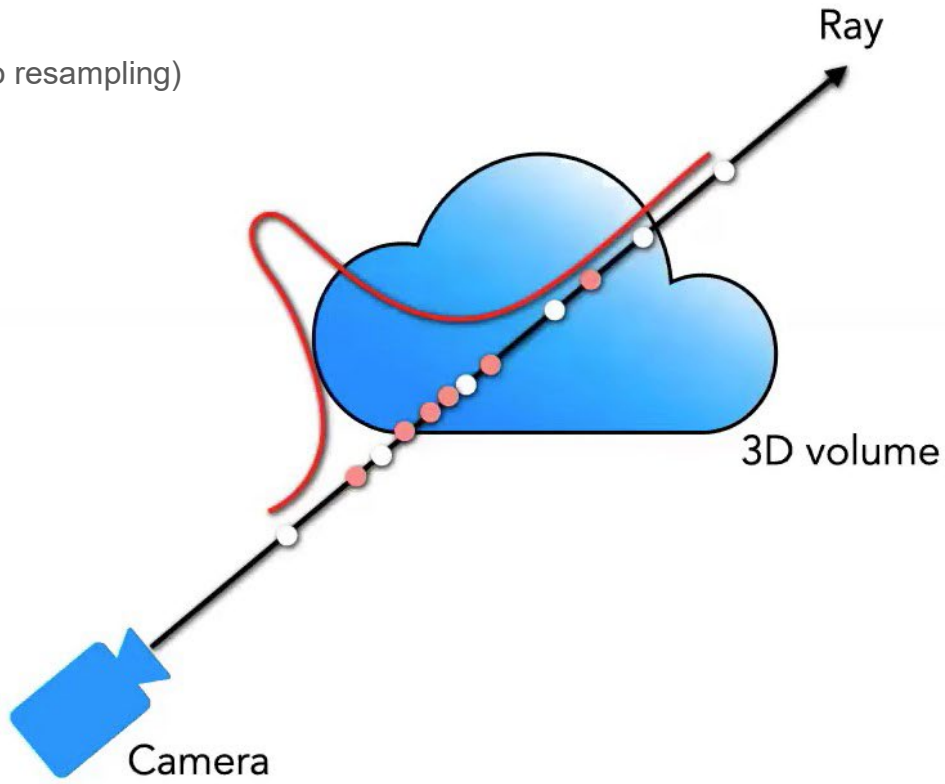
Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Neural Radiance Fields (NeRFs). Importance Sampling

- Key concept: generate **more samples in regions with high opacity or density …**
  - **…** and fewer in empty space.
- At **training** time:
  - Sampling & rendering
- At **inference** time:
  - Used for rendering accumulation (no resampling)

$$C \approx \sum_{i=1}^{N} T_i \alpha_i c_i$$

treat weights as probability distribution for new samples



Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Neural Radiance Fields (NeRFs). Summary

- Represents a scene using a Neural Network (~ volumetric latent space or “fog”)
- Store color and density at each voxel as an MLP
- Renders an image by shooting a ray through this MLP latent space
- Optimizes MLP parameters …
  - … by rendering a set of known viewpoints and comparing to ground truth images.

Source: Roni Sengupta. Lecture 24 on NeRF’s (COMP 590/776: Computer Vision)

Neural Radiance Fields (NeRFs). Limitations

- NeRF learns a **scene-specific radiance field**:
  - You train **one MLP per scene**.
  - **Training** takes hours per scene.
  - **Inference** may also be **slow**
    - It requires multiple inference times of the MLP
  - Requires **ground-truth** information …
    - … for camera/image poses. Generally obtained through SfM.
  - **Do not generalize** to new scenes.
    - After training, it can only render that *one* scene.

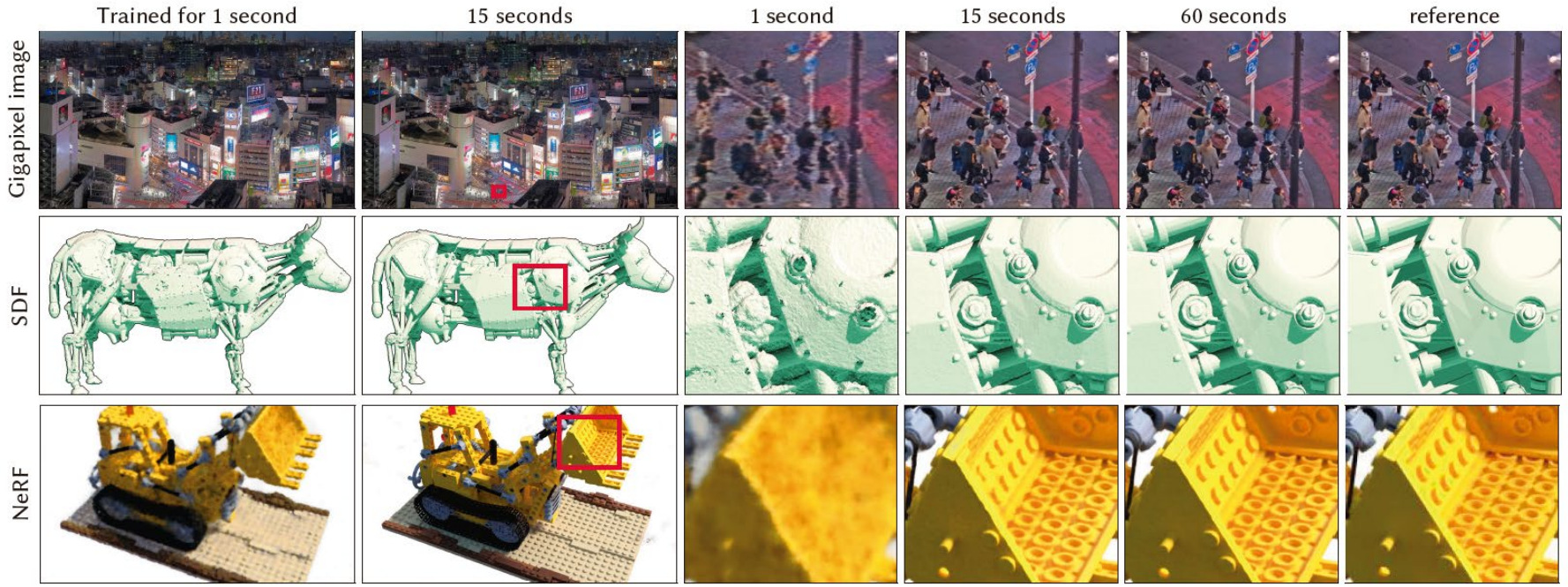
- Thus, standard NeRF's are not so green in terms of AI efficiency.
  - Further research alleviates some of these limitations …

Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Neural Radiance Fields
(Green AI: Towards NeRF Efficient Implementations)

Instant NGP (Neural Graphics Primitives)

- **Reduce slow training times** by leveraging a versatile new input encoding that permits the use of a smaller network (MLP) **without sacrificing quality**.
  - Significantly **reduces the number of floating-point and memory access operations**.



Source: Müller et al. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. ACM Trans. Graph. 41, 4, Article 102 (July 2022)

Instant NGP: Multiresolution Hash Encoding. Key Idea

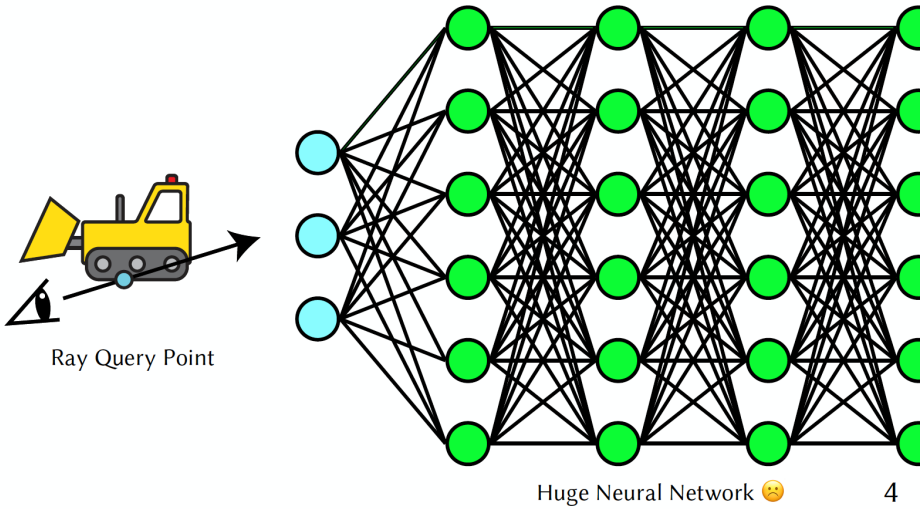
- **Instant-NGP replaces positional encoding (NeRF) with a hash-grid of features.**
  - Each 3D location maps to a set of **learned feature vectors** stored in a table (hash grid).
  - These features are **trainable parameters**, meaning during training they get updated just like MLP weights.
  - Instead of manually encoding inputs (sin/cos), Instant-NGP *learns a better encoding itself*.

- A **hash grid** stores many small feature vectors
  - ➔ More memory than plain positional encoding.
  - ➔ Features directly represent scene information, thus the MLP that follows can be tiny !!!
    - Features are **individual NeRF's trained on small sections of a scene**
    - Tiny MLPs **capture local detail**.
  - ➔ Massively speeds up training and inference compared to NeRF.

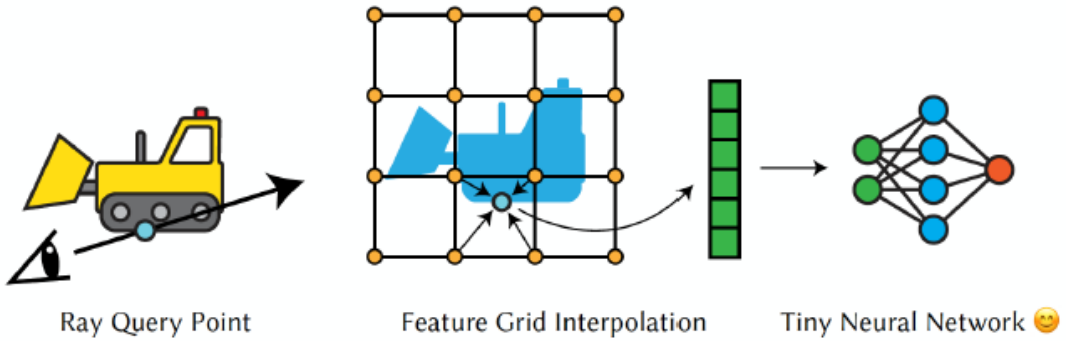
- Use priors obtained from CNNs (e.g. VGG-features)
  - ➔ Used for **generalization**
  - ➔ A vanilla NeRF (and Instant-NGP) usually **memorizes one scene starting from scratch !**

Source: Müller et al. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. ACM Trans. Graph. 41, 4, Article 102 (July 2022)

Instant NGP: Multiresolution Hash Encoding. Key Idea



Versus



Source: Roni Sengupta. Lecture 24 on NeRF's (COMP 590/776: Computer Vision)

Instant NGP: Multiresolution Hash Encoding. How it works?

- **Reduce slow training times** by leveraging a versatile new input encoding that permits the use of a smaller network (MLP) **without sacrificing quality**.
  - Significantly **reduces the number of floating point and memory access operations**.

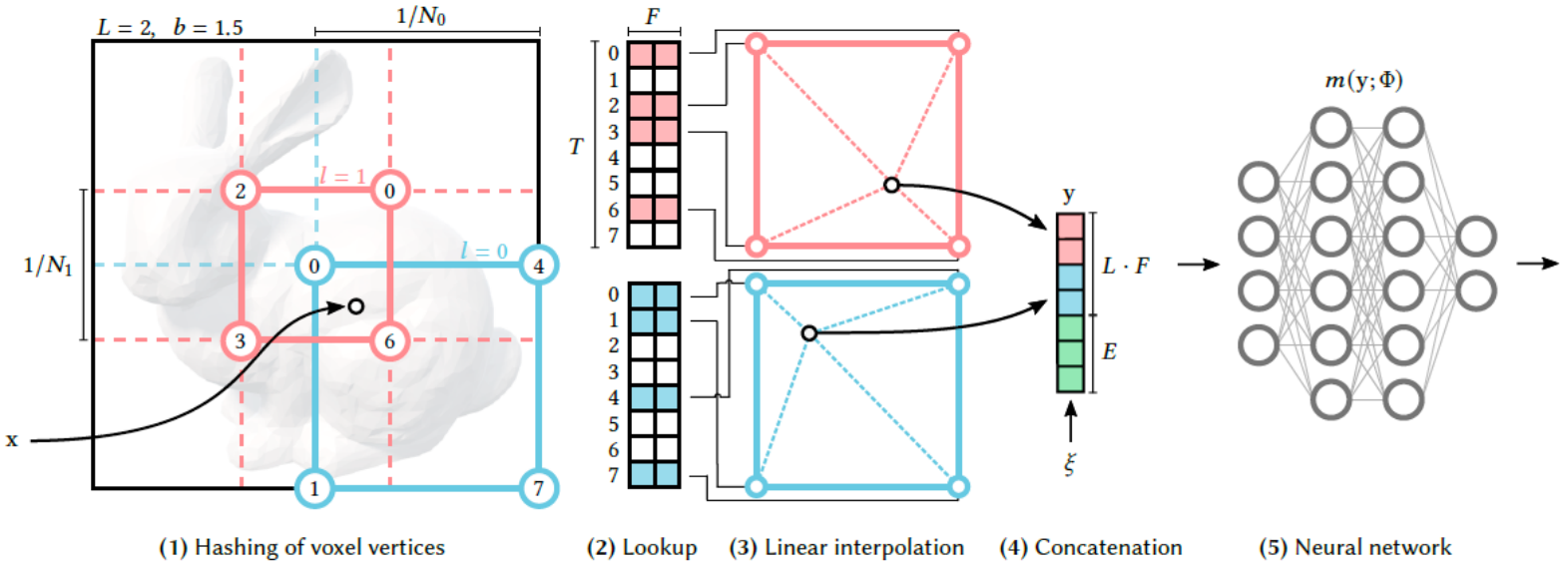


Fig. 3. Illustration of the multiresolution hash encoding in 2D. **(1)** for a given input coordinate x, we find the surrounding voxels at L resolution levels and assign indices to their corners by hashing their integer coordinates. **(2)** for all resulting corner indices, we look up the corresponding F-dimensional feature vectors from the hash tables θ_l and **(3)** linearly interpolate them according to the relative position of x within the respective l-th voxel. **(4)** we concatenate the result of each level, as well as auxiliary inputs $\xi \in \mathbb{R}^E$, producing the encoded MLP input $y \in \mathbb{R}^{LF+E}$, which **(5)** is evaluated last. To train the encoding, loss gradients are backpropagated through the MLP **(5)**, the concatenation **(4)**, the linear interpolation **(3)**, and then accumulated in the looked-up feature vectors.

Source: Müller et al. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. ACM Trans. Graph. 41, 4, Article 102 (July 2022)

Instant NGP: Multiresolution Hash Encoding is Green AI

- **Reduce slow training times** by leveraging a versatile new input encoding that permits the use of a smaller network (MLP) **without sacrificing quality**.
  - Significantly **reduces the number of floating point and memory access operations**.

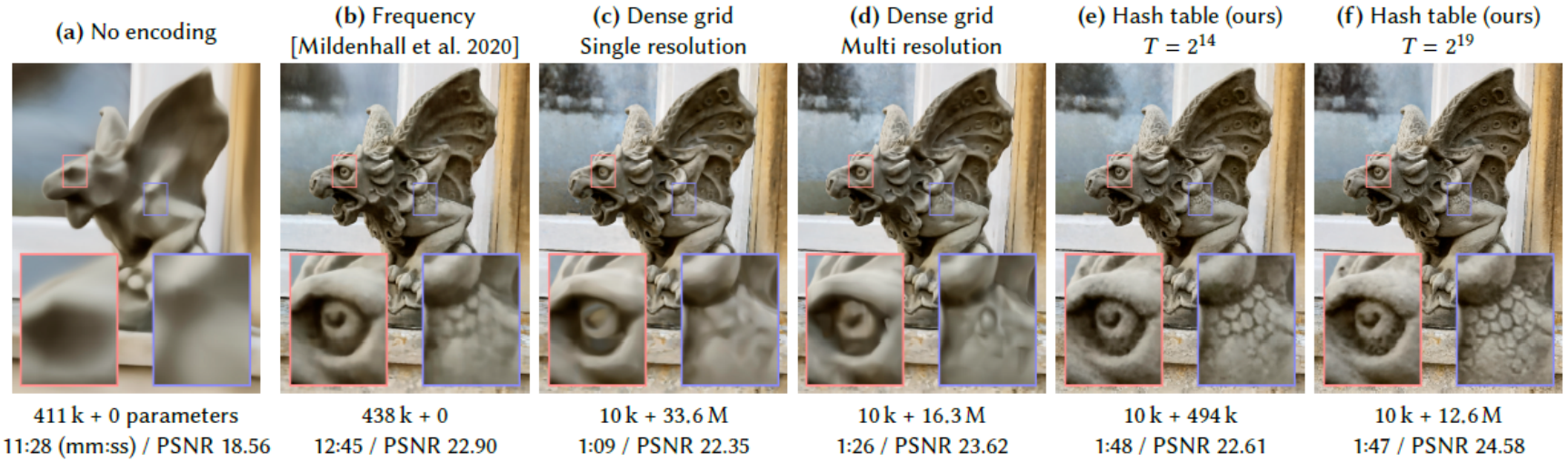


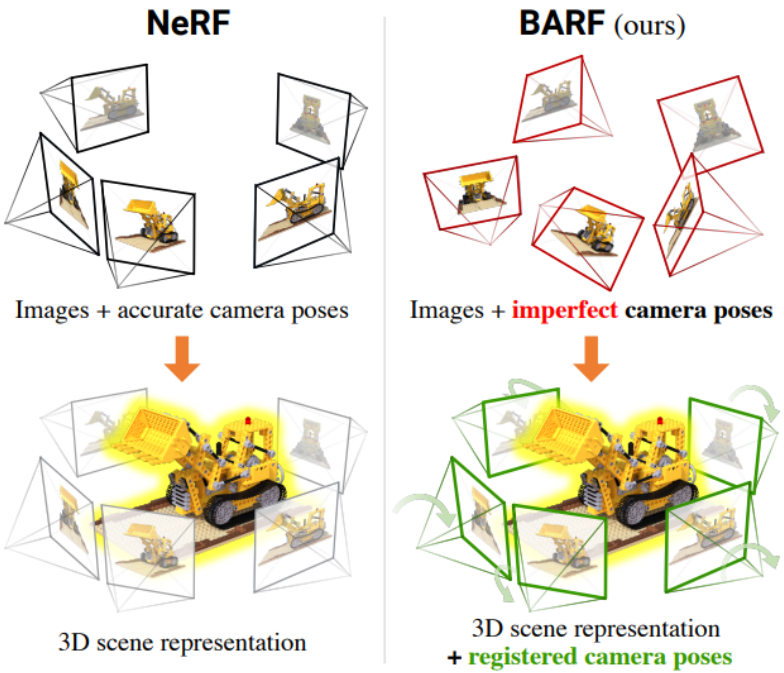
Fig. 2. A demonstration of the reconstruction quality of different encodings and parametric data structures for storing trainable feature embeddings. Each configuration was trained for 11 000 steps using our fast NeRF implementation (Section 5.4), varying only the input encoding and MLP size. The number of trainable parameters (MLP weights + encoding parameters), training time and reconstruction accuracy (PSNR) are shown below each image. Our encoding **(e)** with a similar total number of trainable parameters as the frequency encoding configuration **(b)** trains over 8× faster, due to the sparsity of updates to the parameters and smaller MLP. Increasing the number of parameters **(f)** further improves reconstruction accuracy without significantly increasing training time.

Source: Müller et al. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. ACM Trans. Graph. 41, 4, Article 102 (July 2022)

NeRF working with Imperfect Pose Estimation

Bundle-Adjusting Neural Radiance Fields

- Attacks the limitation of requiring **accurate camera poses** to learn the scene representation (e.g. SfM & SLAM).
- Propose a **coarse-to-fine registration for NeRF** on coordinate-based scene representations.



- A simple yet effective strategy for training NeRF from imperfect camera poses
- Learn coarse-to-fine signals by weighting the k-th frequency component of γ as:

$$\gamma_k(\mathbf{x};\alpha) = w_k(\alpha)\cdot\big[\cos(2^k\pi\mathbf{x}),\sin(2^k\pi\mathbf{x})\big]\ ,$$

where the weight w_k is defined as

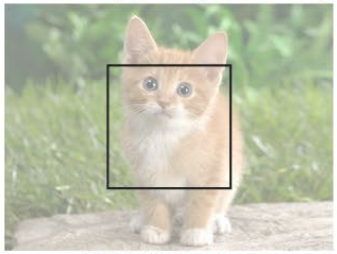
$$w_k(\alpha) = \begin{cases} 0 & \text{if } \alpha < k \\ \dfrac{1-\cos((\alpha-k)\pi)}{2} & \text{if } 0 \le \alpha - k < 1 \\ 1 & \text{if } \alpha - k \ge 1 \end{cases}$$

- And $\alpha \in [0, L]$ is a controllable parameter proportional to the optimization progress.
- We gradually activate the encodings of higher frequency bands until full positional encoding is enabled ($\alpha = \mathrm{L}$).

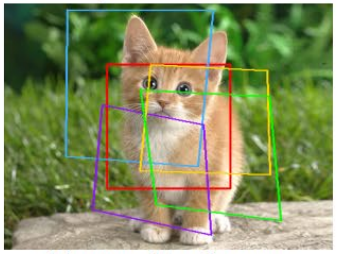
https://chenhsuanlin.bitbucket.io/bundle-adjusting-NeRF/

Bundle-Adjusting Neural Radiance Fields. Planar Image Alignment (2D)

(a) image patches given for optimization



(b) initialization



(c) ground-truth warps

Figure 3: Given image patches color-coded in (a), we aim to recover the alignment *and* the neural representation of the entire image, with the patches initialized to center crops shown in (b) and the corresponding ground-truth warps shown in (c).

positional encoding	$\mathfrak{sl}(3)$ error	patch PSNR
naïve (full)	0.2949	23.41
without	0.0641	24.72
BARF (coarse-to-fine)	0.0096	35.30

Table 1: **Quantitative results** of planar image alignment. BARF optimizes for more accurate alignment and patch reconstruction compared to the baselines.

(a) naïve pos. enc. (b) w/o pos. enc. (c) BARF

Figure 4: **Qualitative results** of the planar image alignment experiment. We visualize the optimized warps (top row), the patch reconstructions in corresponding colors (middle row), and recovered image representation from f (bottom row). BARF is able to recover accurate alignment and high-fidelity image reconstruction, while baselines result in suboptimal alignment with naïve positional encoding and blurry reconstruction without any encoding. Best viewed in color.

Source: Lin et al. BARF : Bundle-Adjusting Neural Radiance Fields (ICCV, 2021).

Bundle-Adjusting Neural Radiance Fields. Planar Image Alignment (3D)

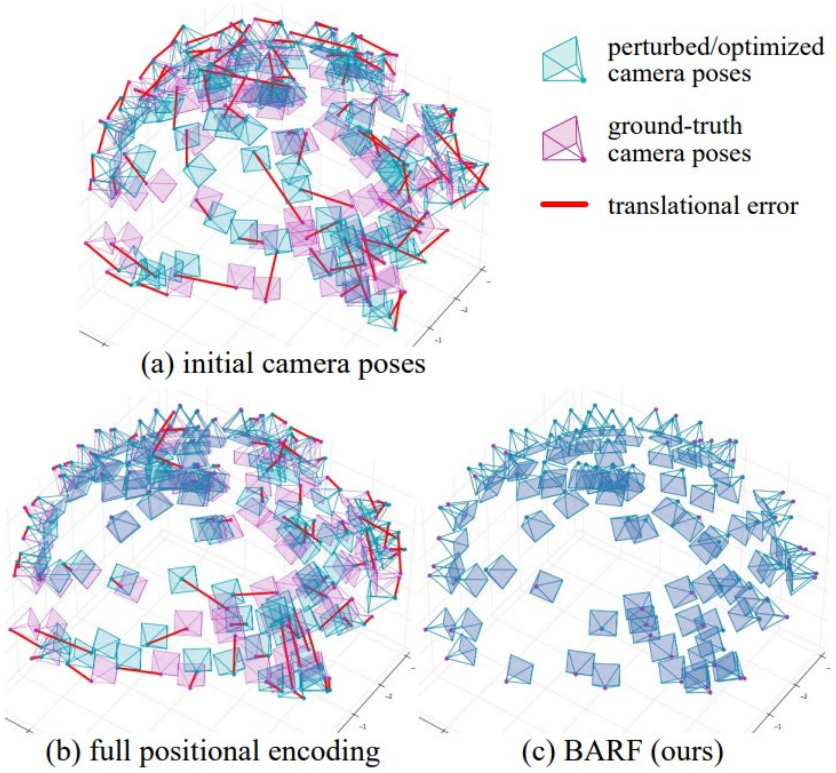


Figure 5: Visual comparison of the initial and optimized camera poses (Procrustes aligned) for the *chair* scene. BARF successfully realigns all the camera frames while NeRF naïve positional encoding gets stuck at suboptimal solutions.

Source: https://chenhsuanlin.bitbucket.io/bundle-adjusting-NeRF/

Generalization for NeRF's

pixelNeRF: Neural Radiance Fields from One or Few Images

- Addressing generalization issues …

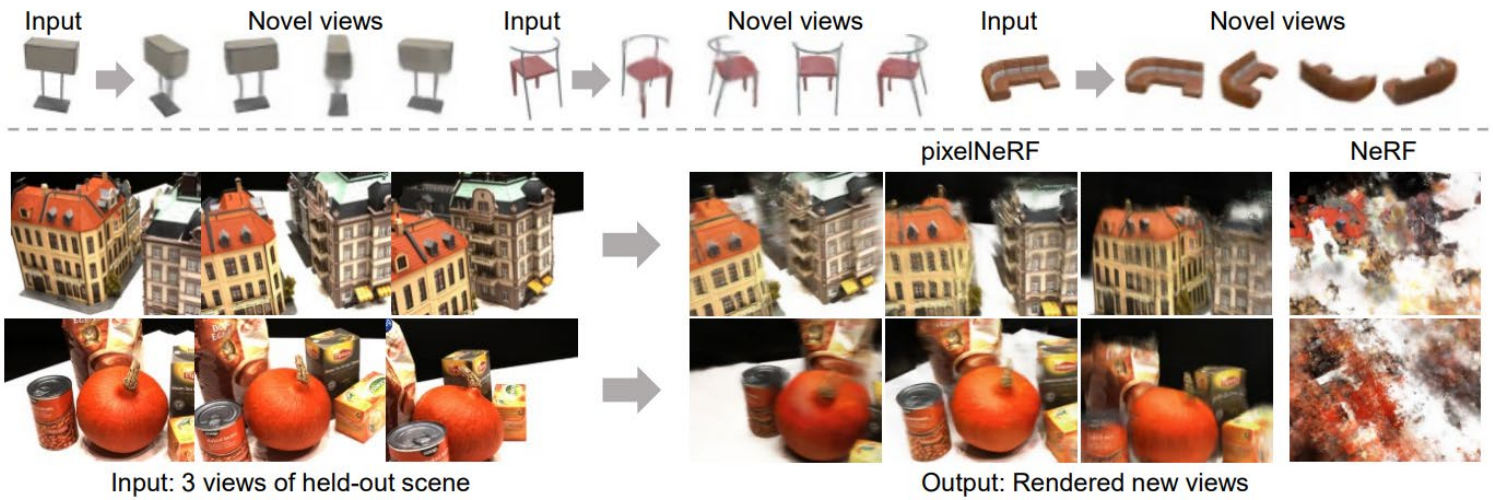


Figure 1: **NeRF from one or few images.** We present pixelNeRF, a learning framework that predicts a Neural Radiance Field (NeRF) representation from a single (top) or few posed images (bottom). PixelNeRF can be trained on a set of multi-view images, allowing it to generate plausible novel view synthesis from very few input images without test-time optimization (bottom left). In contrast, NeRF has no generalization capabilities and performs poorly when only three input views are available (bottom right).

Source: Yu, Alex, et al. "pixelnerf: Neural radiance fields from one or few images." *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021.

pixelNeRF: Neural Radiance Fields from One or Few Images

- **Single-Image pixelNeRF** injects the feature vector $\boldsymbol{W}(\pi \boldsymbol{x})$, where $\boldsymbol{W}$ is a feature volume computed from the input single-image $\boldsymbol{I}$.
- Perhaps more intuitively : *"they inject multiview consistency by reprojecting original image features that match new generated views* $\{ (x, d) \}$ *".*

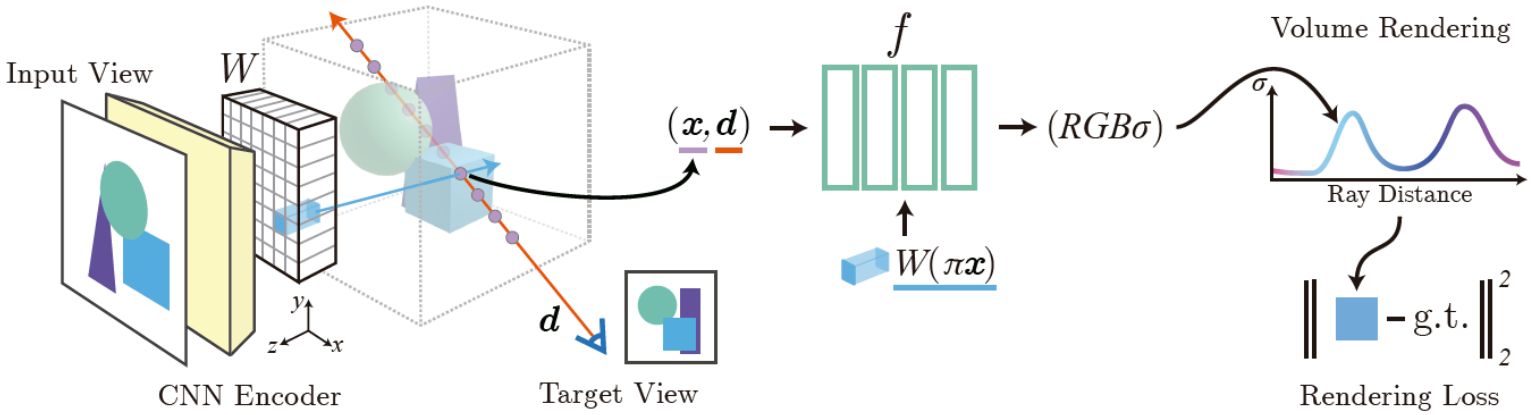


Figure 2: **Proposed architecture in the single-view case.** For a query point $\mathbf{x}$ along a target camera ray with view direction $\mathbf{d}$, a corresponding image feature is extracted from the feature volume $\mathbf{W}$ via projection and interpolation. This feature is then passed into the NeRF network f along with the spatial coordinates. The output RGB and density value is volume-rendered and compared with the target pixel value. The coordinates $\mathbf{x}$ and $\mathbf{d}$ are in the camera coordinate system of the input view.

Source: Yu, Alex, et al. "pixelnerf: Neural radiance fields from one or few images." *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021.

pixelNeRF: Neural Radiance Fields from One or Few Images

- **Multiview Images (pixelNeRF)**
  - Camera transforms for each camera:

$$\mathbf{P}^{(i)} = \begin{bmatrix} \mathbf{R}^{(i)} & \mathbf{t}^{(i)} \end{bmatrix}$$

  - Transform a query point x, with view direction d, into the coordinate system of each view i:

$$\mathbf{x}^{(i)} = \mathbf{P}^{(i)}\mathbf{x}, \quad \mathbf{d}^{(i)} = \mathbf{R}^{(i)}\mathbf{d}$$

  - Pass the inputs to obtain intermediate vectors to f_1 (first layers)

$$\mathbf{V}^{(i)} = f_1\left(\gamma(\mathbf{x}^{(i)}), \mathbf{d}^{(i)}; \mathbf{W}^{(i)}\left(\pi(\mathbf{x}^{(i)})\right)\right)$$

  - And finally aggregated using average pooling operator and passed into f_2 (final layers) to obtain the final color and density:

$$(\sigma, \mathbf{c}) = f_2\left(\psi\left(\mathbf{V}^{(1)}, \ldots, \mathbf{V}^{(n)}\right)\right)$$

Source: Yu, Alex, et al. "pixelnerf: Neural radiance fields from one or few images." *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021.

pixelNeRF: Neural Radiance Fields from One or Few Images

- Novel-view synthesis results on a real image dataset (3 views as input).

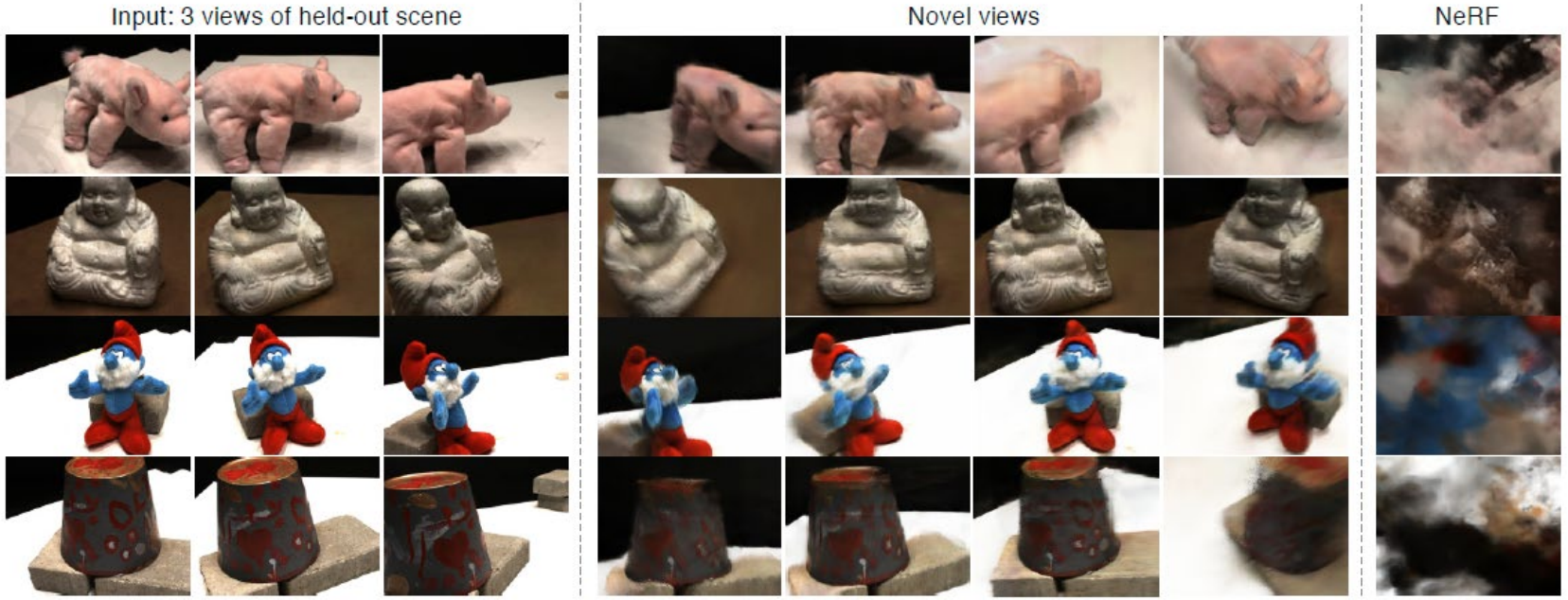


Figure 9: **Wide baseline novel-view synthesis on a real image dataset.** We train our model to distinct scenes in the DTU MVS dataset [14]. Perhaps surprisingly, even in this case, our model is able to infer novel views with reasonable quality for held-out scenes without further test-time optimization, all from only three views. Note the train/test sets share no overlapping scenes.

Source: Yu, Alex, et al. "pixelnerf: Neural radiance fields from one or few images." *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021.

3D Gaussian Splatting (3DGS)

Problems with NeRF

- Training efficiency is very slow (~ several hours)
- Rendering is also rather slow (barely near interactive at high resolutions)
- Why? $\#images \times \#pixels \times \#points$
- For every sample, it needs to run NN inference

3D Gaussian Splatting

This video contains a voice-over

3D Gaussian Splatting for Real-Time Radiance Field Rendering

SIGGRAPH 2023
(ACM Transactions on Graphics)

Bernhard Kerbl*
Inria
UNIVERSITÉ CÔTE D'AZUR

Georgios Kopanas*
Inria
UNIVERSITÉ CÔTE D'AZUR

Thomas Leimkühler
mpii
max planck institut informatik

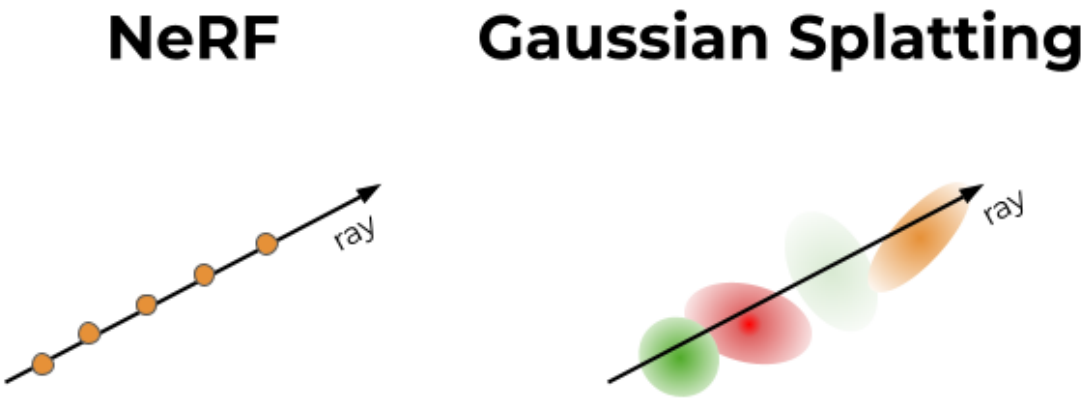
George Drettakis
Inria
UNIVERSITÉ CÔTE D'AZUR

* Denotes equal contribution

GraphDeco
Graphics and Design with Heterogeneous COntent

3D Gaussian Splatting

- Achieve state-of-the-art visual quality
- Competitive training times
- Allow high-quality real-time rendering (> 30 fps) at 1080p

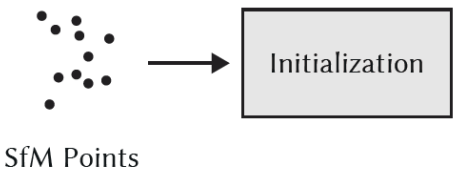


3D Gaussian Splatting for Real-time Radiance Field Rendering

- General pipeline overview

3D Gaussian Splatting for Real-time Radiance Field Rendering

- General pipeline overview
- First, we initialize camera poses with Structure from Motion (SfM)

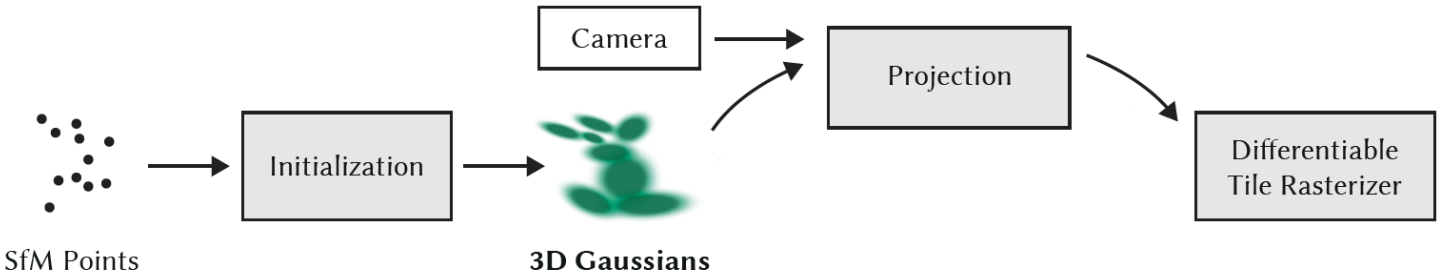


3D Gaussian Splatting for Real-time Radiance Field Rendering

- General pipeline overview
- Using the sparse 3D structure initialize a set of initial 3D gaussians

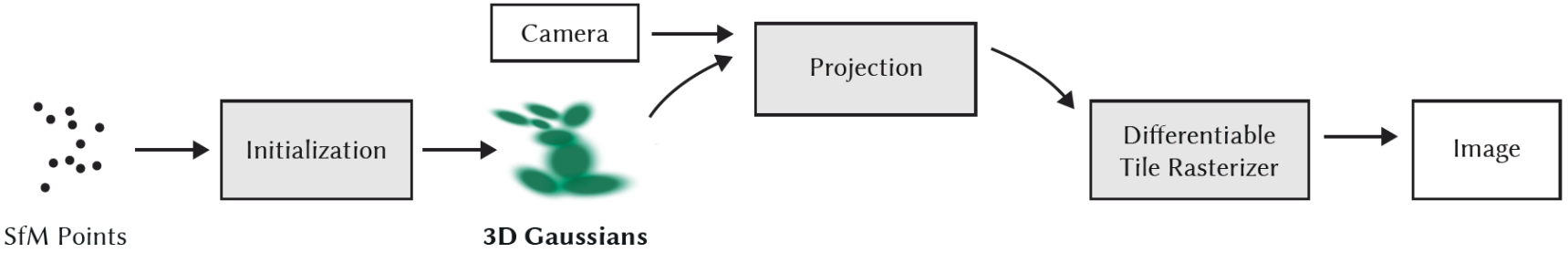
3D Gaussian Splatting for Real-time Radiance Field Rendering

- General pipeline overview
- Given a camera view, project/render the 3D gaussians using a Tile-based Differential Rasterizer



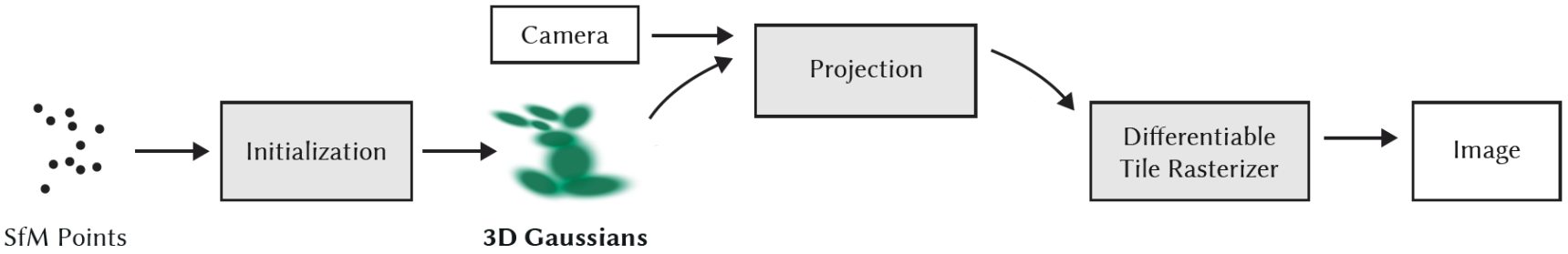
3D Gaussian Splatting for Real-time Radiance Field Rendering

- General pipeline overview
- Generate an estimated image



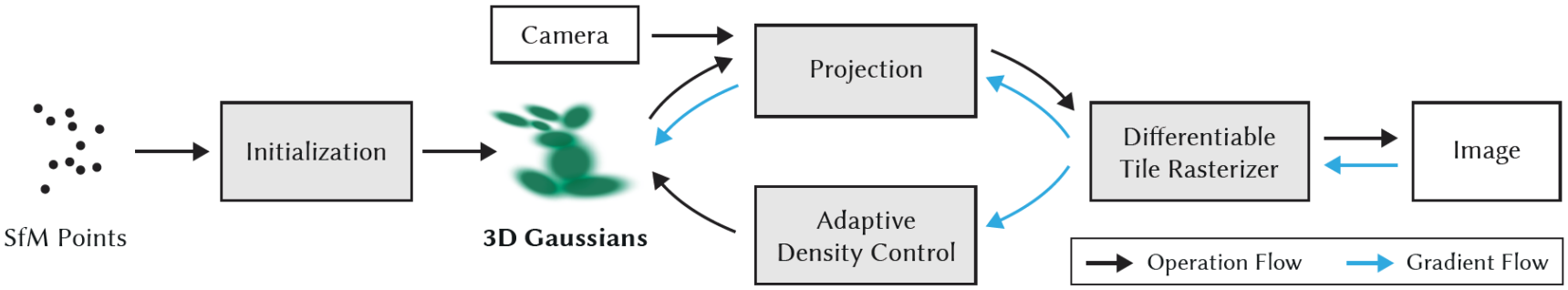
3D Gaussian Splatting for Real-time Radiance Field Rendering

- General pipeline overview



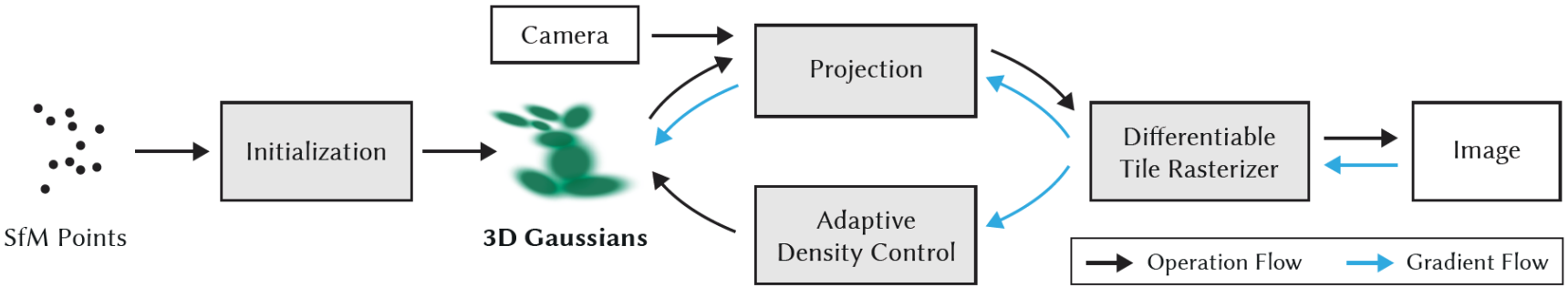
3D Gaussian Splatting for Real-time Radiance Field Rendering

- General pipeline overview
- Backpropagate the error gradient from the rendered image towards the parameters of the 3D gaussians, so they are progressively improved.



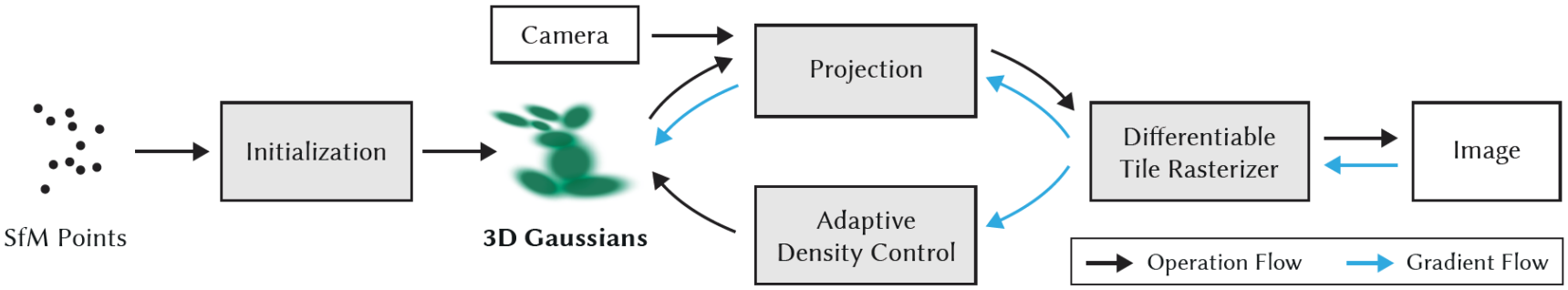
3D Gaussian Splatting for Real-time Radiance Field Rendering

- General pipeline overview
- Optionally, the pipeline permits to adaptively control the density of 3d gaussians, removeing or adding gaussians where needed to improve the final representation.



3D Gaussian Splatting for Real-time Radiance Field Rendering

- General pipeline overview
- Summary.
  - No NN involved! ➔ much faster training and rendering!
  - Just **modern machine learning** + classic **CV** methods (SfM) + classic 3D **CG** (Rasterization)

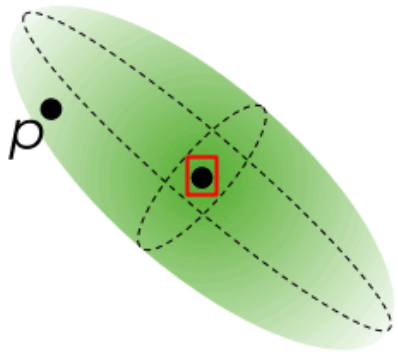


3D Gaussians

- Center position μ_i
- Covariance 3×3 matrix Σ_i (Shape)
- Density / Opacity $\sigma(\alpha_i)$
- Color *(r,g,b)*
- ***p***, point where we want to evaluate the 3d gaussian representation

$$\Sigma = RSS^T R^T$$

$$f_i(p) = \sigma(\alpha_i) \exp(-\frac{1}{2}(p - \mu_i)\Sigma_i^{-1}(p - \mu_i))$$



Source: A Comprehensive Overview of Gaussian Splatting | Towards Data Science

3D Gaussians

This video contains a voice-over

3D Gaussian Splatting for Real-Time Radiance Field Rendering

SIGGRAPH 2023
(ACM Transactions on Graphics)

Bernhard Kerbl*
Inria
UNIVERSITÉ CÔTE D'AZUR

Georgios Kopanas*
Inria
UNIVERSITÉ CÔTE D'AZUR

Thomas Leimkühler
mpii
max planck institut informatik

George Drettakis
Inria
UNIVERSITÉ CÔTE D'AZUR

* Denotes equal contribution

GraphDeco
Graphics and Design with Heterogeneous Content

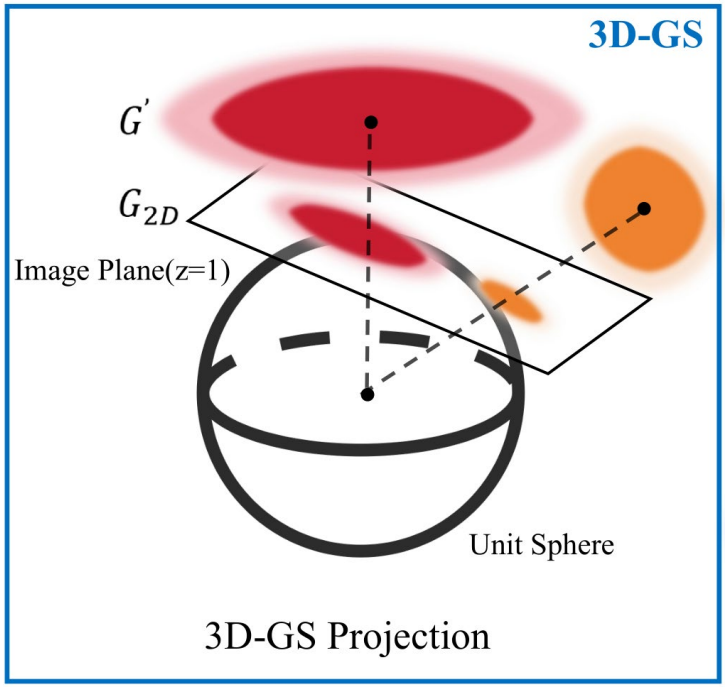
3D Gaussians. Initialization

- e.g. from Structure from Motion (SfM) …

Source: A Comprehensive Overview of Gaussian Splatting | Towards Data Science

3D Gaussians. Projection

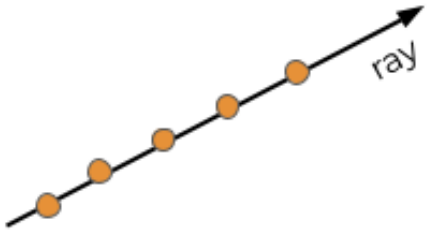
- From a set of 3d gaussians
  - Project mean and covariance into a 2D plane ➔ easy to compute impact into a certain pixel.



Source: A Comprehensive Overview of Gaussian Splatting | Towards Data Science

3D Gaussians. Rendering

- NeRF image formation model
  - **Queries a continuous MLP** along the ray.



$$\sum_{i=1}^{N} c_i \alpha_i \underbrace{\prod_{j=1}^{i-1}(1-\alpha_j)}_{transmittance}$$

- 3DGS uses the projection $f_i^{2\mathrm{D}}(p)$ of 3d gaussians into 2D:
  - i.e. image plane of the camera that is being rendered.
  - **Blends a discrete set of Gaussians** relevant to the given ray.

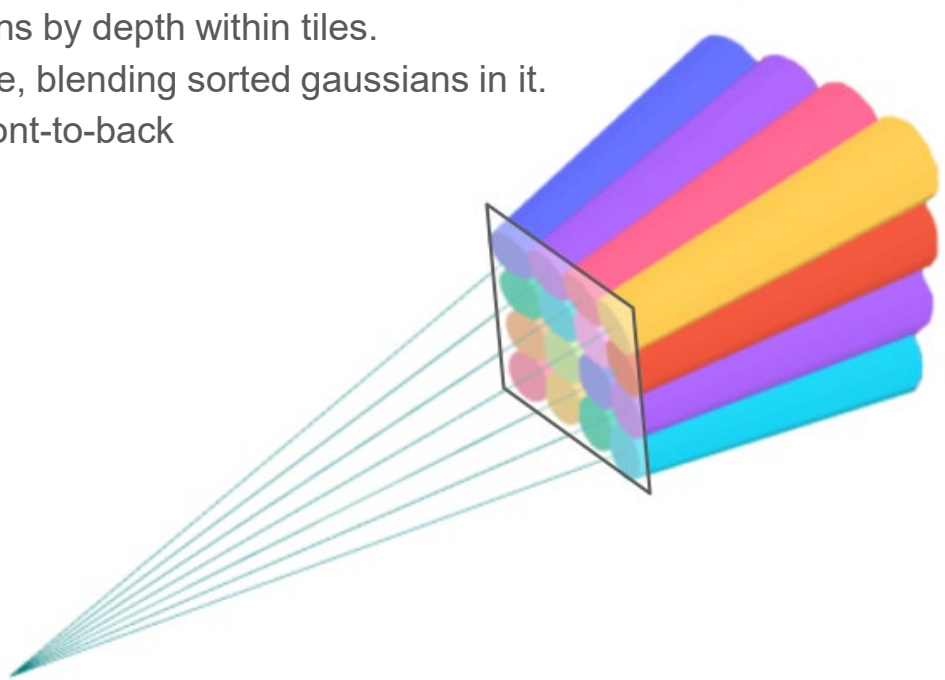
$$C(p) = \sum_{i \in N} c_i f_i^{2D}(p) \underbrace{\prod_{j=1}^{i-1}(1 - f_j^{2D}(p))}_{transmittance}$$

Source: A Comprehensive Overview of Gaussian Splatting | Towards Data Science

3D Gaussians. Sorting

- Sort gaussians by depth to determine transmittance (radix/bucket sort)
  - Sorting depths for Gaussian splatting is like sorting decimal numbers by digits

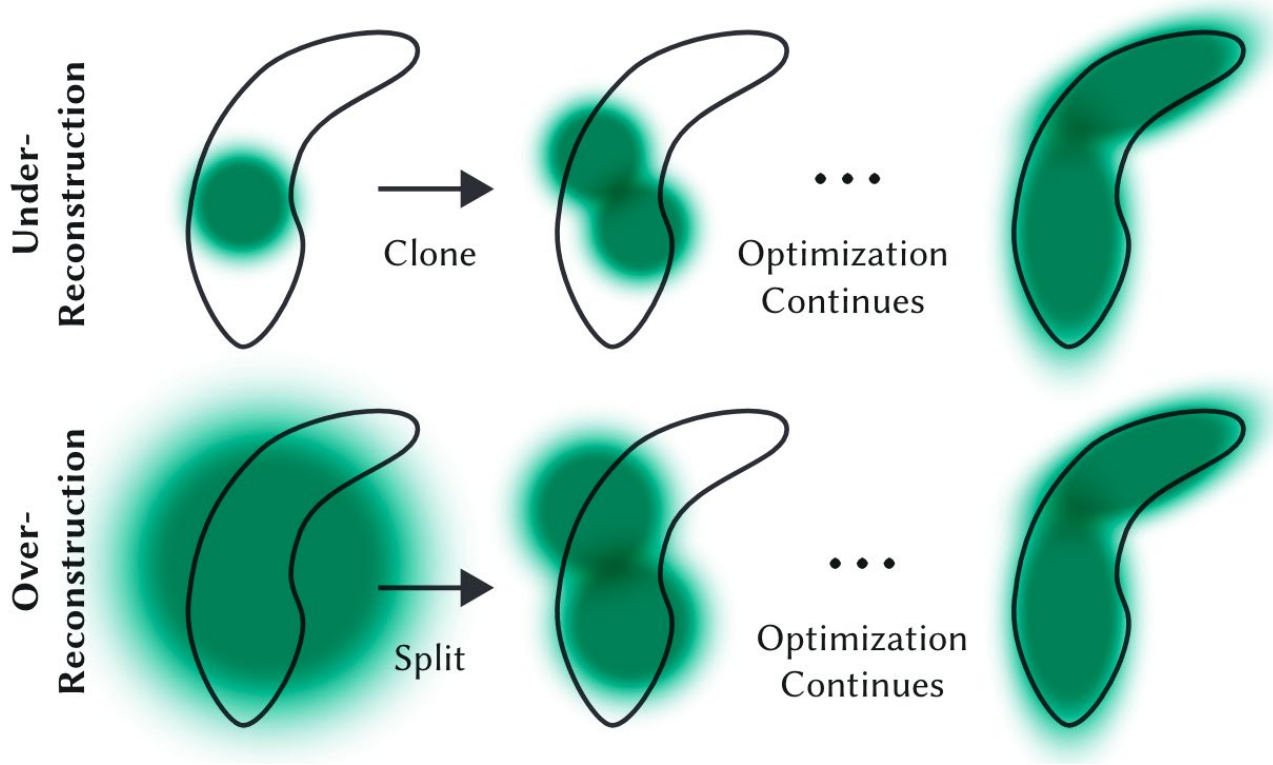
- View frustums, each corresponding to a 16×16 pixels image tile (4x4 pixels)
  - **Assign** gaussians to tiles.
  - **Sort** gaussians by depth within tiles.
  - **Render** by tile, blending sorted gaussians in it.
    - E.g. Front-to-back



Source: https://docs.nerf.studio/nerfology/model_components/visualize_samples.html#d-frustum

3D Gaussians. Adaptive Density Control

- Clone & Split operations …

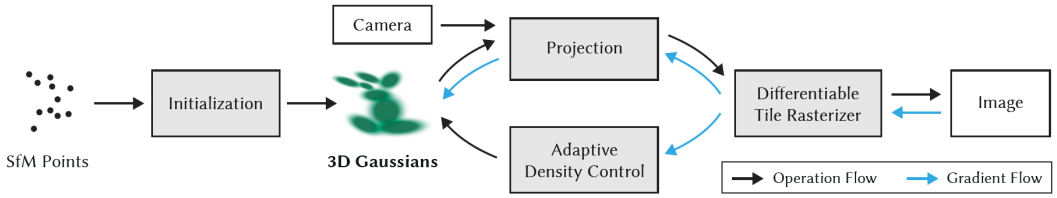


Source: A Comprehensive Overview of Gaussian Splatting | Towards Data Science

3D Gaussian Splatting. Summary

Algorithm 1 Optimization and Densification
w, h: width and height of the training images

$M \leftarrow$ SfM Points ▷ Positions
$S, C, A \leftarrow$ InitAttributes() ▷ Covariances, Colors, Opacities
$i \leftarrow 0$ ▷ Iteration Count
while not converged **do**
  $V, \hat{I} \leftarrow$ SampleTrainingView() ▷ Camera V and Image
  $I \leftarrow$ Rasterize(M, S, C, A, V) ▷ Alg. 2
  $L \leftarrow Loss(I, \hat{I})$ ▷ Loss
  $M, S, C, A \leftarrow$ Adam(∇L) ▷ Backprop & Step
  if IsRefinementIteration(i) **then**
    for all Gaussians (μ, Σ, c, α) **in** (M, S, C, A) **do**
      if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning
        RemoveGaussian()
      end if
      if $\nabla_p L > \tau_p$ **then** ▷ Densification
        if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction
          SplitGaussian(μ, Σ, c, α)
        else ▷ Under-reconstruction
          CloneGaussian(μ, Σ, c, α)
        end if
      end if
    end for
  end if
  $i \leftarrow i + 1$
end while



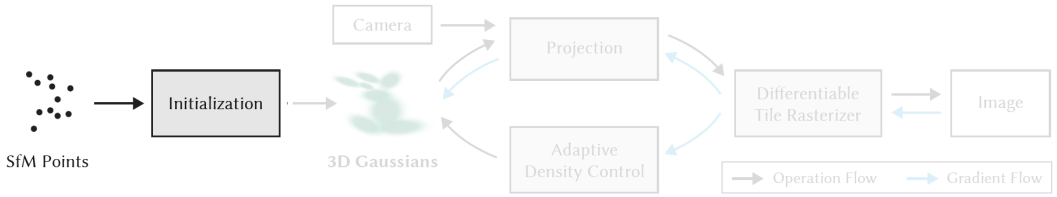
Source: A Comprehensive Overview of Gaussian Splatting | Towards Data Science

3D Gaussian Splatting. Summary

Algorithm 1 Optimization and Densification

w, h: width and height of the training images

$M \leftarrow$ SfM Points ▷ Positions
$S, C, A \leftarrow$ InitAttributes() ▷ Covariances, Colors, Opacities
$i \leftarrow 0$ ▷ Iteration Count
while not converged do
  $V, \hat{I} \leftarrow$ SampleTrainingView() ▷ Camera V and Image
  $I \leftarrow$ Rasterize(M, S, C, A, V) ▷ Alg. 2
  $L \leftarrow Loss(I, \hat{I})$ ▷ Loss
  $M, S, C, A \leftarrow$ Adam(∇L) ▷ Backprop & Step
  if IsRefinementIteration(i) then
    for all Gaussians (μ, Σ, c, α) in (M, S, C, A) do
      if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) then ▷ Pruning
        RemoveGaussian()
      end if
      if $\nabla_p L > \tau_p$ then ▷ Densification
        if $\|S\| > \tau_S$ then ▷ Over-reconstruction
          SplitGaussian(μ, Σ, c, α)
        else ▷ Under-reconstruction
          CloneGaussian(μ, Σ, c, α)
        end if
      end if
    end for
  end if
  $i \leftarrow i + 1$
end while

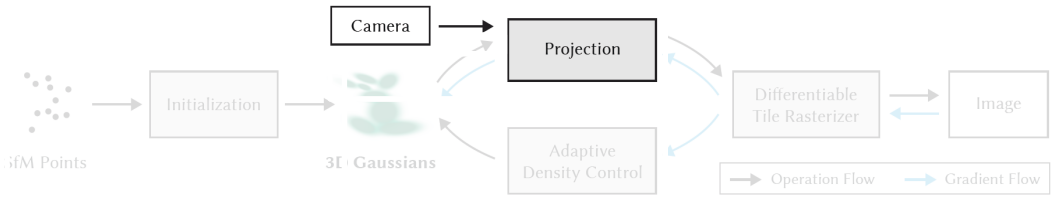


Source: A Comprehensive Overview of Gaussian Splatting | Towards Data Science

3D Gaussian Splatting. Summary

Algorithm 1 Optimization and Densification
w, h: width and height of the training images

$M \leftarrow$ SfM Points ▷ Positions
$S, C, A \leftarrow$ InitAttributes() ▷ Covariances, Colors, Opacities
$i \leftarrow 0$ ▷ Iteration Count
while not converged do
  $V, \hat{I} \leftarrow$ SampleTrainingView() ▷ Camera V and Image
  $I \leftarrow$ Rasterize(M, S, C, A, V) ▷ Alg. 2
  $L \leftarrow Loss(I, \hat{I})$ ▷ Loss
  $M, S, C, A \leftarrow$ Adam(∇L) ▷ Backprop & Step
  if IsRefinementIteration(i) **then**
    for all Gaussians (μ, Σ, c, α) in (M, S, C, A) **do**
      if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning
        RemoveGaussian()
      end if
      if $\nabla_p L > \tau_p$ **then** ▷ Densification
        if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction
          SplitGaussian(μ, Σ, c, α)
        else ▷ Under-reconstruction
          CloneGaussian(μ, Σ, c, α)
        end if
      end if
    end for
  end if
  $i \leftarrow i + 1$
end while

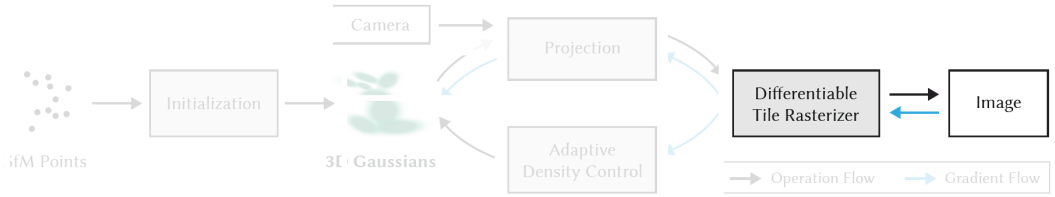


Source: A Comprehensive Overview of Gaussian Splatting | Towards Data Science

3D Gaussian Splatting. Summary

Algorithm 1 Optimization and Densification
w, h: width and height of the training images

$M \leftarrow$ SfM Points ▷ Positions
$S, C, A \leftarrow$ InitAttributes() ▷ Covariances, Colors, Opacities
$i \leftarrow 0$ ▷ Iteration Count
while not converged **do**
  $V, \hat{I} \leftarrow$ SampleTrainingView() ▷ Camera V and Image
  $I \leftarrow$ Rasterize(M, S, C, A, V) ▷ Alg. 2
  $L \leftarrow Loss(I, \hat{I})$ ▷ Loss
  $M, S, C, A \leftarrow$ Adam(∇L) ▷ Backprop & Step
  if IsRefinementIteration(i) **then**
    for all Gaussians (μ, Σ, c, α) in (M, S, C, A) **do**
      if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning
        RemoveGaussian()
      end if
      if $\nabla_p L > \tau_p$ **then** ▷ Densification
        if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction
          SplitGaussian(μ, Σ, c, α)
        else ▷ Under-reconstruction
          CloneGaussian(μ, Σ, c, α)
        end if
      end if
    end for
  end if
  $i \leftarrow i + 1$
end while



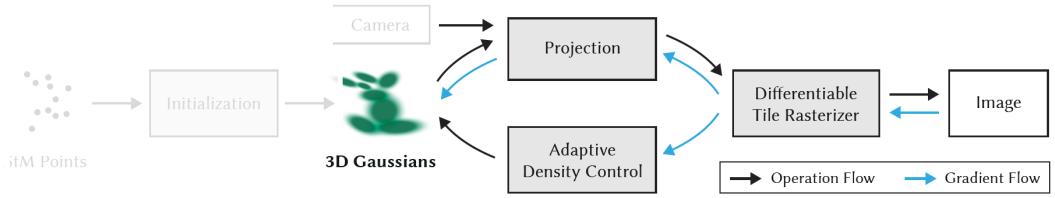
Source: A Comprehensive Overview of Gaussian Splatting | Towards Data Science

3D Gaussian Splatting. Summary

Algorithm 1 Optimization and Densification

w, h: width and height of the training images

$M \leftarrow$ SfM Points ▷ Positions
$S, C, A \leftarrow$ InitAttributes() ▷ Covariances, Colors, Opacities
$i \leftarrow 0$ ▷ Iteration Count
while not converged do
  $V, \hat{I} \leftarrow$ SampleTrainingView() ▷ Camera V and Image
  $I \leftarrow$ Rasterize(M, S, C, A, V) ▷ Alg. 2
  $L \leftarrow Loss(I, \hat{I})$ ▷ Loss
  $M, S, C, A \leftarrow$ Adam(∇L) ▷ Backprop & Step
  if IsRefinementIteration(i) then
    for all Gaussians (μ, Σ, c, α) in (M, S, C, A) do
      if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) then ▷ Pruning
        RemoveGaussian()
      end if
      if $\nabla_p L > \tau_p$ then ▷ Densification
        if $\|S\| > \tau_S$ then ▷ Over-reconstruction
          SplitGaussian(μ, Σ, c, α)
        else ▷ Under-reconstruction
          CloneGaussian(μ, Σ, c, α)
        end if
      end if
    end for
  end if
  $i \leftarrow i + 1$
end while

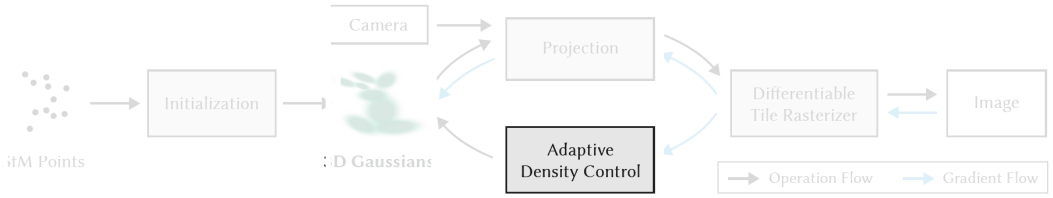


Source: A Comprehensive Overview of Gaussian Splatting | Towards Data Science

3D Gaussian Splatting. Summary

Algorithm 1 Optimization and Densification
w, h: width and height of the training images

$M \leftarrow$ SfM Points ▷ Positions
$S, C, A \leftarrow$ InitAttributes() ▷ Covariances, Colors, Opacities
$i \leftarrow 0$ ▷ Iteration Count
while not converged do
  $V, \hat{I} \leftarrow$ SampleTrainingView() ▷ Camera V and Image
  $I \leftarrow$ Rasterize(M, S, C, A, V) ▷ Alg. 2
  $L \leftarrow Loss(I, \hat{I})$ ▷ Loss
  $M, S, C, A \leftarrow$ Adam(∇L) ▷ Backprop & Step
  if IsRefinementIteration(i) **then**
    for all Gaussians (μ, Σ, c, α) **in** (M, S, C, A) **do**
      if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning
        RemoveGaussian()
      end if
      if $\nabla_p L > \tau_p$ **then** ▷ Densification
        if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction
          SplitGaussian(μ, Σ, c, α)
        else ▷ Under-reconstruction
          CloneGaussian(μ, Σ, c, α)
        end if
      end if
    end for
  end if
  $i \leftarrow i + 1$
end while

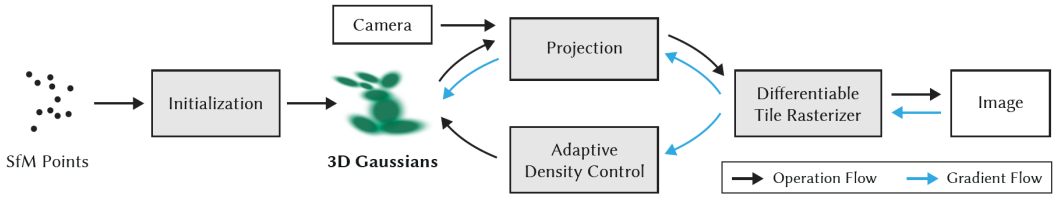


Source: A Comprehensive Overview of Gaussian Splatting | Towards Data Science

3D Gaussian Splatting. Summary

Algorithm 1 Optimization and Densification
w, h: width and height of the training images

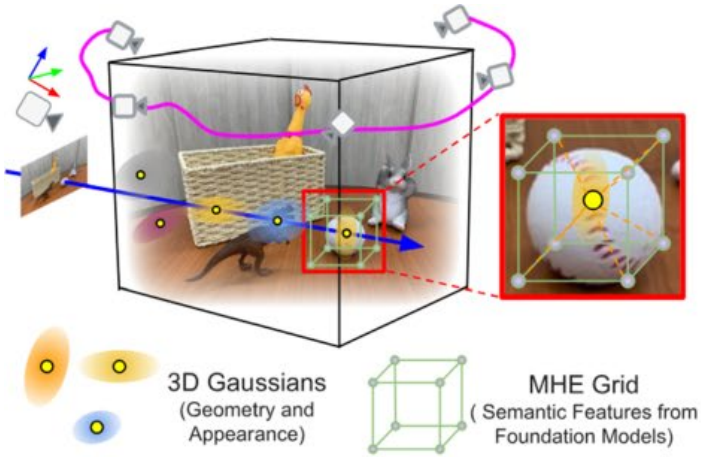
$M \leftarrow$ SfM Points ▷ Positions
$S, C, A \leftarrow$ InitAttributes() ▷ Covariances, Colors, Opacities
$i \leftarrow 0$ ▷ Iteration Count
while not converged **do**
  $V, \hat{I} \leftarrow$ SampleTrainingView() ▷ Camera V and Image
  $I \leftarrow$ Rasterize(M, S, C, A, V) ▷ Alg. 2
  $L \leftarrow Loss(I, \hat{I})$ ▷ Loss
  $M, S, C, A \leftarrow$ Adam(∇L) ▷ Backprop & Step
  if IsRefinementIteration(i) **then**
    for all Gaussians (μ, Σ, c, α) **in** (M, S, C, A) **do**
      if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning
        RemoveGaussian()
      end if
      if $\nabla_p L > \tau_p$ **then** ▷ Densification
        if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction
          SplitGaussian(μ, Σ, c, α)
        else ▷ Under-reconstruction
          CloneGaussian(μ, Σ, c, α)
        end if
      end if
    end for
  end if
  $i \leftarrow i + 1$
end while



Source: A Comprehensive Overview of Gaussian Splatting | Towards Data Science

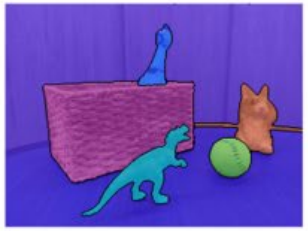
State-of-the-art in 3DGS is vastly expanding …

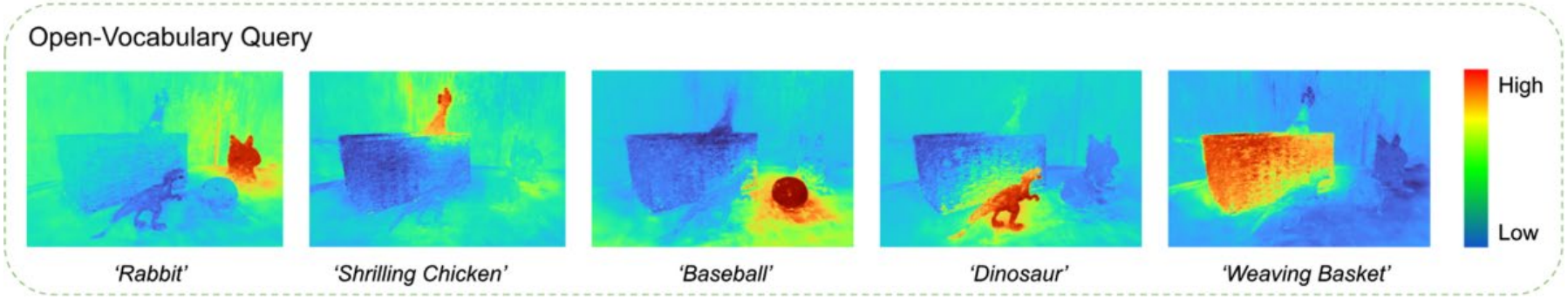
- There is vast work going on in 3DGS + AI Embedded Foundation Models …
  - e.g. Foundation Model Embedded 3D Gaussian Splatting for Holistic 3D Scene Understanding




Rendered RGB


Rendered CLIP Feature


Unsupervised Semantic Segmentation



Source: Zuo et al. FMGS: Foundation Model Embedded 3D Gaussian Splatting for Holistic 3D Scene Understanding, 2024

Nerfstudio: A practical tutorial for prototyping 3D-IBR

- We will briefly comment the nerfstudio workflow and main characteristics

INDITEX UDC

Cátedra de IA en Algoritmos Verdes

GRACIAS POR SU ATENCIÓN

https://catedrainditexalgoritmosverdes.com/

MICROCREDENCIAL ALGORITMOS VERDES PARA LA IA: DISEÑO E IMPLEMENTACIÓN

Tema: AI, Graphics & Visual Computing: Usos de la IA para la Reconstrucción 3D. Generación de Datos Sintéticos.

Sesión: Lunes 6 de octubre de 2025

Ponente: José A. Iglesias Guitián

citic

UNIVERSIDADE DA CORUÑA

Financiado por la Unión Europea NextGenerationEU

GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA

ENIA ESTRATEGIA NACIONAL DE INTELIGENCIA ARTIFICIAL

España | digital 2026